Manual Funcional del Sistema — Modular Learning

Versión del documento: 2.0
Alcance: Especificación funcional completa basada en el comportamiento real del código fuente (`backend/`, `frontend/`, `scripts/`, `deploy/`), contrastada con la documentación del repositorio (`README.md`, `docs/ARCHITECTURE.md`, `docs/DESPLIEGUE.md`) y los criterios funcionales esperados (`docs/PENDIENTES.md` — tarea D1).

---

1. Qué es y para quién

1.1. Propósito del sistema

Modular Learning es un asistente académico e interactivo de estudio universitario diseñado bajo **Arquitectura Hexagonal (Puertos y Adaptadores)** e impulsado por **Google Gemini** (`google-genai`, modelo por defecto `gemini-3.5-flash-lite` definido en `backend/src/modular_learning/infrastructure/llm/gemini_adapter.py:27`). Su objetivo es acompañar al estudiante durante todo el semestre en seis actividades centrales:

1. **Planificación curricular:** organizar el catálogo de asignaturas y extraer cronogramas de clases desde sílabos oficiales (`.pdf` o `.docx`).
2. **Ingesta y alineación de material:** convertir lecturas, diapositivas, guías de actividad y proyectos prácticos en apuntes estructurados en Markdown dentro de una bóveda de **Obsidian**, vinculándolos automáticamente a la sesión correspondiente del cronograma.
3. **Tutoría socrática interactiva:** guiar el estudio de un tema paso a paso mediante explicaciones en *streaming* (Server-Sent Events), resolución de dudas con contexto curricular y práctico, y generación de un resumen final en Obsidian.
4. **Estudio fuera de línea (Guías Offline):** generar cuadernos de estudio en formato **PDF A5** listos para imprimir o leer sin conexión, y registrar al regreso qué clases quedaron entendidas o con dudas.
5. **Evaluación activa y repaso espaciado:** generar cuestionarios (*quizzes*) de opción múltiple y preguntas abiertas evaluadas con IA, programando tarjetas de repaso bajo una variante del algoritmo **SM-2**.
6. **Contexto práctico mediante plugins:** analizar laboratorios y código fuente de proyectos reales (**Cisco Packet Tracer**, **Node-RED**, **Desarrollo Web**) para inyectar su arquitectura y estado en las explicaciones del tutor.

1.2. Destinatarios de este manual

- **Estudiante (usuario final):** utiliza la interfaz web o la consola para gestionar su semestre, ingestar apuntes, estudiar con el tutor o en papel y repasar antes de los parciales.
- **Revisor técnico** / **mantenedor:** verifica qué hace exactamente cada vista, endpoint y comando del sistema, dónde persiste cada dato y qué discrepancias existen entre el comportamiento esperado y la implementación actual.

1.3. Interfaces del sistema

El núcleo de aplicación (`backend/src/modular_learning/core/`) se expone a través de dos adaptadores primarios que comparten el mismo contenedor de dependencias (`Contenedor` en `backend/src/modular_learning/bootstrap.py:42-151`):

1. **Aplicación Web (`ml-web` + SPA React):**
   - **Backend HTTP:** servidor FastAPI (`backend/src/modular_learning/apps/api/main.py:29-141`) iniciado con el entrypoint `ml-web` (`pyproject.toml:34`, puerto `8000` por defecto). Expone 55 rutas bajo `/api/*` y sirve la SPA compilada si existe `frontend/dist` (`apps/api/main.py:113-132`).
   - **Frontend SPA:** aplicación React 19 + TypeScript + Vite (`frontend/src/App.tsx:78-401`) organizada en una barra lateral con tres bloques principales (**Hoy**, **Estudiar**, **Asignaturas**) y un pie de utilidades (**Plugins**, **Progreso**, **Ajustes**).
2. **Interfaz de Línea de Comandos (`ml`):**
   - **CLI con Typer y Rich:** definida en `backend/src/modular_learning/apps/cli/main.py:10-70` (`pyproject.toml:33`). Ofrece los subcomandos `ml init`, `ml es`, `ml im`, `ml quiz`, `ml stats` y `ml up`, además del módulo ejecutable `apps/cli/commands/snapshot.py` y los scripts operativos `scripts/migrar_datos.py` y `deploy/subir_datos.py`.

1.4. Modos de operación: Local, Nube e Híbrido

El comportamiento de persistencia, seguridad y acceso al sistema de archivos depende de las variables de entorno leídas en `bootstrap.py:44-56`, `apps/api/routers/config.py:21` y `apps/api/acceso.py:19-31`:

Característica	Modo Local	Modo en la Nube (Render + Neon)	Modo Híbrido (PC local + Neon)
Variables clave	Sin `DATABASE_URL`; sin `ML_EN_NUBE`	`DATABASE_URL=postgresql://...`, `ML_EN_NUBE=1`, `ML_REQUIERE_ACCESO=1`	`DATABASE_URL=postgresql://...`, sin `ML_EN_NUBE` (o `ML_EN_NUBE=0`)

Almacén de documentos (`AlmacenDocumentos`)	`AlmacenArchivos` (`infrastructure/persistence/almacen.py:32-70`) sobre `data/` (o `ML_DATA_DIR`)	`AlmacenPostgres` (`infrastructure/persistence/almacen.py:73-147`) en la tabla `documentos` de Neon	`AlmacenPostgres` conectado a la misma tabla `documentos` de Neon
Acceso a carpetas locales de sílabos, material y proyectos	Sí (`LectorDocumentosLocal`, `InspectorProyectosLocal`, plugins)	No tiene acceso al disco del PC del estudiante; las rutas en el contenedor apuntan a `/tmp/ml-boveda` y `/app/data/material` (`deploy/subir_datos.py:23-29`)	Sí: lee los archivos locales del PC mientras lee/escribe el estado JSON en Neon
Bóveda de Obsidian	Escribe directamente en la carpeta local configurada en `obsidian_vault`	Escribe en `/tmp/ml-boveda` dentro del contenedor (efímero entre reinicios de Render)	Escribe en la bóveda local de Obsidian si `obsidian_vault` apunta a una ruta local válida
Autenticación HTTP	Desactivada por defecto (`ML_REQUIERE_ACCESO` no activo)	HTTP Basic obligatorio en todas las rutas salvo `/api/salud` (`apps/api/acceso.py:34-70`) usando `ML_USUARIO` y `ML_CONTRASENA`	Opcional (habitualmente desactivada en `localhost`)
Interfaz de Configuración (`ConfigView`)	Permite editar rutas locales y clave de Gemini (`frontend/src/views/ConfigView.tsx:221-334`)	Oculta el formulario de rutas/API key y muestra un aviso de gestión por variables de entorno (`ConfigView.tsx:214-220`)	Permite editar ajustes (cuidado: pisa `config/ajustes.json` en Postgres si se guarda)

2. Conceptos del dominio

2.1. Asignatura (`Materia`: `slug` y `nombre`)

- **Definición:** entidad `Materia(slug: str, nombre: str)` en `backend/src/modular_learning/core/domain/materias.py:42-55`.
- **Cálculo del `slug`:** la función `slug_materia(texto)` (`core/domain/materias.py:11-30`) descompone caracteres Unicode (`NFKD`), elimina marcas diacríticas (tildes/diéresis), convierte a minúsculas y reemplaza cualquier secuencia no alfanumérica por un guion `-`. Es idempotente: `slug_materia("Ingeniería de Software II") == slug_materia("ingenieria-de-software-ii") == "ingenieria-de-software-ii"`.
- **En el frontend:** `clave(texto)` en `frontend/src/lib/asignaturas.ts:13-20` aplica la misma normalización para cruzar nombres con el catálogo (`buscarMateriaPorClave`, `nombreEnCatalogo`).
- **Propósito:** el `slug` es la clave primaria e invariante para nombrar archivos de cronogramas (`schedules/<slug>_cronograma.json`) y referenciar la asignatura en URLs; el `nombre` es la etiqueta legible con mayúsculas y tildes que se muestra en la interfaz y nombra las carpetas dentro de Obsidian.

2.2. Cronograma

- **Definición:** documento JSON por asignatura almacenado bajo la clave `schedules/<slug>_cronograma.json` (`infrastructure/persistence/json_repositorios.py:107-111`).
- **Estructura:** contiene `materia` (nombre visible), `clases` (lista ordenada de diccionarios de clase), `parciales` (lista opcional de cortes de parcial) y `material_transversal` (lista opcional de nombres de materiales que aplican a toda la asignatura).

2.3. Clase (`id` y `estado`)

- **Definición:** cada entrada de la lista `clases` de un cronograma (`core/domain/cronograma.py:8-66`).
- **Identificador (`id`):** cadena hexadecimal de 8 caracteres generada con `uuid.uuid4().hex[:8]` por `nuevo_id_clase()` (`core/domain/cronograma.py:11-12`). `asegurar_ids_clases()` (`cronograma.py:15-31`) garantiza que toda clase tenga un `id` único dentro del cronograma al cargar o guardar.
- **Atributos:**
  - `id`: identificador único de la sesión.
  - `tema`: título del tema de la clase.
  - `fecha`: cadena con la fecha o semana (ej. `"14/04/2026"` o `"Semana 1"`).
  - `objetivo`: competencia u objetivo pedagógico de la sesión.
  - `contenido`: lista de subtemas (`list[str]`) o cadena descriptiva.
  - `estado`: `"pendiente"` (por defecto) o `"completado"`.
  - `fecha_completado`: fecha ISO (`YYYY-MM-DD`) añadida cuando la clase pasa a `"completado"` (`json_repositorios.py:249-250`).
  - `materiales_asociados`: lista de nombres de archivos ingestados vinculados a esta clase por el alineador curricular (`core/domain/alineador_curricular.py:92-96`).
  - `subtemas_adicionales`: lista de subtemas extraídos de los materiales ingestados e inyectados en la clase (`alineador_curricular.py:98-103`).

2.4. Parcial (Corte de parcial)

- **Definición:** marcador dentro del campo `parciales` del cronograma (`[{"nombre": "Parcial 1", "hasta": "<referencia>"}]`), gestionado por `JsonCronogramaRepo.obtener_parciales` y `guardar_parciales` (`json_repositorios.py:293-315`).
- **Propósito:** delimitar hasta qué clase abarca cada examen parcial para permitir seleccionar de un clic todas las clases pendientes de ese corte al armar una guía offline (`frontend/src/views/GuiaView.tsx:341-376`). *(Véase la discrepancia DISC-10 sobre el formato del campo `hasta` entre `SilabosView` y `GuiaView`).*

2.5. Material de estudio y alineación curricular

- **Definición:** archivo académico procesado por `IngestaService` (`core/application/ingesta.py:23-323`).
- **Formatos admitidos en el código actual:** `.pdf`, `.docx`, `.txt` y `.md` (`EXTENSIONES_MATERIAL` en `ingesta.py:23`). Los archivos `.pptx` están declarados en `EXTENSIONES_NO_SOPORTADAS_WEB = (".pptx",)` (`ingesta.py:24`) y se rechazan o se omiten como no soportados *(véase DISC-05 / P16)*.
- **Clasificación automática por ruta:** `IngestaService._clasificar_archivo` (`ingesta.py:55-68`) inspecciona el nombre de la subcarpeta donde reside el archivo:
  - Contiene `"actividad"` o `"taller"` → tipo `actividad` → subcarpeta Obsidian `Actividades`.
  - Contiene `"diapositiva"` o `"presentacion"` → tipo `diapositiva` → subcarpeta Obsidian `Diapositivas`.
  - Contiene `"complementario"` → tipo `material_complementario` → subcarpeta Obsidian `Complementario`.
  - Cualquier otro caso (o subida suelta por web) → tipo `lectura` → subcarpeta Obsidian `Lecturas`.
- **Alineación curricular:** al ingestar un archivo, `IngestaService._extraer_metadatos` (`ingesta.py:86-102`) pide a Gemini que identifique `tema_principal`, `subtemas`, `es_transversal` y la `clase_asociada`. Luego `asignar_material_a_clase` (`core/domain/alineador_curricular.py:51-109`):
  - Si `es_transversal` es `True`, agrega el archivo a `cronograma["material_transversal"]`.
  - Si no, calcula la similitud léxica (`calcular_similitud` con `SequenceMatcher` + intersección de palabras de más de 3 letras, `alineador_curricular.py:13-48`) contra todas las clases del cronograma. Si la mejor coincidencia alcanza o supera el umbral `0.6` (60%), vincula el archivo en `materiales_asociados` e inyecta sus `subtemas_adicionales` en esa clase.

2.6. Nota en Obsidian

- **Definición:** archivo Markdown (`.md`) escrito en el sistema de archivos local a través del puerto `BovedaObsidianPort` (`infrastructure/obsidian/boveda_obsidian.py:13-133`).
- **Tipos de notas generadas:**
  1. **Apunte de material ingestado:** `<obsidian_vault>/<materia>/<SubcarpetaTipo>/<nombre_base>_Notas.md` (`ingesta.py:131-134`).
  2. **Ficha técnica de proyecto:** `<obsidian_vault>/<materia>/Proyectos/<nombre_limpio>.md` (`proyectos.py:141-145`).
  3. **Resumen de sesión de tutoría:** `<obsidian_vault>/<materia>/<tema_limpio>.md` con *frontmatter* YAML (`tags`, `fecha`, `estado: completado`) y enlace bidireccional en el índice `<obsidian_vault>/<materia>.md` (`boveda_obsidian.py:44-119`).

2.7. Proyecto

- **Definición:** entidad `Proyecto` (`core/domain/proyectos.py:33-65`) que representa un entregable práctico o documental de varias semanas (un repositorio de código, una topología IoT o un informe de auditoría) ubicado dentro de `<material_dir>/<Asignatura>/Proyectos/...`.
- **Detección e inspección:** `InspectorProyectosLocal` (`infrastructure/proyectos/inspector_local.py:44-183`) detecta carpetas de proyecto saltando directorios contenedores (`proyectos`, `general`, `primer_parcial`, `segundo_parcial`, etc.) y clasifica cada proyecto en uno de cuatro tipos (`detectar_tipo`, líneas 129-149):
  - `software`: contiene archivos `.py`, `.ts`, `.tsx`, `.js`, `.java`, `.cs`, `.go` o `.sql`.
  - `iot`: contiene flujos `.json` o archivos `.ino`.
  - `documental_auditoria`: contiene más de un `.docx`/`.pdf` o su ruta incluye `"auditoria"` / `"informe"`.
  - `documento_rector`: caso por defecto cuando solo contiene un documento base.
- **Parcial asociado:** deducido de los nombres de carpeta en la ruta (`inferir_parcial_desde_ruta`, `core/domain/proyectos.py:76-94`): `"primer_parcial"`, `"segundo_parcial"` o `"general"`.
- **Huella de cambio (`HuellaProyecto`):** almacena `total_archivos` y `ultimo_mtime` (`core/domain/proyectos.py:14-30`). Se considera modificado cuando cambia el número de archivos o cuando `ultimo_mtime` supera el registrado por más de `TOLERANCIA_MTIME_SEGUNDOS = 2.0` segundos (`proyectos.py:11`).

2.8. Guía offline

- **Definición:** entidad `GuiaOffline` (`core/domain/guia_offline.py:92-181`) compuesta por una lista ordenada de `ClaseGuia` (`guia_offline.py:44-89`). Permite estudiar un conjunto de clases fuera de la pantalla (impresa en cuadernillo A5 o en lector PDF).
- **Nivel de extensión (`ExtensionGuia`):** determina la profundidad del prompt enviado a Gemini (`core/application/guias.py:21-38`) y se elige automáticamente (`elegir_extension`, `guia_offline.py:194-213`):
  - Por presupuesto de tiempo (`minutos_disponibles`): `< 45` min → `"sintesis"`, `< 90` min → `"estandar"`, `>= 90` min → `"completa"`.
  - Por cantidad de clases seleccionadas (si no se indican minutos): `1-2` clases → `"completa"` (desarrollo completo + 3 preguntas),

`3-5` clases → `"estandar"` (350-500 palabras + 2 preguntas), `6+` clases → `"sintesis"` (200-300 palabras + 1 pregunta).

- **Separación de respuestas:** cada clase generada incluye el marcador `<!-- RESPUESTAS -->` (`MARCADOR_RESPUESTAS`, `guia_offline.py:14`). Al exportar a PDF o imprimir, el cuerpo de estudio (`cuerpo_estudio`) se coloca en la sección de lectura y el bloque de soluciones (`respuestas`) se agrupa al final del documento en un «Solucionario» (`infrastructure/pdf/exportador_pdf.py:67-86`).
- **Ciclo de revisión y reversión:** cada `ClaseGuia` guarda `estado_previo` (el estado que tenía la clase en el cronograma antes de confirmar la guía) y `resultado` (`"pendiente"`, `"entendida"`, `"con_dudas"`). Si se marca `"entendida"`, la clase pasa a `"completado"` en el cronograma y se registra como dominada en el repaso espaciado; si se deshace la revisión, cada clase vuelve a su `estado_previo` (`core/application/guias.py:157-250`).

2.9. Repaso espaciado y tarjeta (`Tarjeta`)

- **Definición:** entidad `Tarjeta` (`core/domain/repaso_espaciado.py:21-62`) generada a partir de cada pregunta respondida en un quiz o al cerrar una tutoría / guía offline.
- **Identificador determinista:** `id_tarjeta(materia, tema, pregunta)` calcula los primeros 12 caracteres del hash SHA-1 de `slug_materia(materia)::tema.lower()::pregunta.lower()` (`repaso_espaciado.py:14-18`), evitando duplicar tarjetas para la misma pregunta.
- **Algoritmo de programación (`programar`):** implementado en `core/domain/repaso_espaciado.py:74-99` (variante SM-2):
  - **Fallo (`acerto = False`):** `repeticiones = 0`, `intervalo_dias = 1` (vence mañana), `facilidad = max(1.3, facilidad - 0.2)`, `fallos += 1`.
  - **Acierto (`acerto = True`):** incrementa `repeticiones` y `aciertos`; si `repeticiones == 1` → `intervalo_dias = 1`; si `repeticiones == 2` → `intervalo_dias = 3`; si `repeticiones >= 3` → `intervalo_dias = max(4, round(intervalo_anterior * facilidad))` y `facilidad = min(2.8, facilidad + 0.05)`.
  - **Tarjeta dominada:** cuando `intervalo_dias >= INTERVALO_DOMINADA` (`21` días, línea 10), `Tarjeta.dominada` es `True` (`repaso_espaciado.py:39-40`).
  - **Límite diario:** `LIMITE_REPASO_DIARIO = 10` preguntas por sesión (`repaso_espaciado.py:11`), priorizando las tarjetas con mayor número de `fallos` y fecha `proximo_repaso` más antigua (`core/application/repaso.py:124`).

2.10. Plugin práctico

- **Definición:** adaptador que implementa `PluginPort` (`core/ports/plugin_port.py`) para extraer contexto técnico de herramientas de laboratorio y guardarlo como memoria Markdown en `plugins/<id>_context.md`.
- **Plugins incluidos en el sistema (`bootstrap.py:117-122`):**
  1. `cisco` (`CiscoPlugin` en `infrastructure/plugins/cisco.py:16-303`): descomprime y analiza archivos `.pkt` / `.pka` de Cisco Packet Tracer (con `zlib` y heurísticas binarias/XML) y documentos `.pdf`/`.docx` adjuntos para extraer dispositivos, VLANs, direccionamiento IP y rúbricas. Materia por defecto: `"Conmutación y Enrutamiento"`.
  2. `nodered` (`NodeRedPlugin` en `infrastructure/plugins/nodered.py:14-215`): analiza flujos JSON de Node-RED identificando nodos MQTT, HTTP, funciones JavaScript y dashboards. Materia por defecto: `"Internet de las Cosas"`.
  3. `web` (`WebPlugin` en `infrastructure/plugins/web.py:332-442`): inspecciona proyectos full-stack (`.py`, `.ts`, `.tsx`, `.js`, `.html`, `.css`, `package.json`, `requirements.txt`), resumiendo su árbol de directorios, stack tecnológico y esqueleto de clases/funciones. Materia por defecto: `"Ingeniería de Software II"`.
- **Enrutamiento al tutor (`PluginsService.obtener_plugin_para_materia`):** implementado en `core/application/plugins.py:125-175`, busca coincidencia en tres niveles: 1) configuración explícita en `config/plugins_config.json`, 2) `materia_asociada_defecto` del plugin (comparando tanto nombre normalizado como `slug_materia`), y 3) palabras clave en el nombre de la asignatura o del tema.

---

3. Datos y almacenamiento

3.1. Abstracción `AlmacenDocumentos`

Toda la persistencia de configuración y estado del sistema pasa por el puerto `AlmacenDocumentos` (`infrastructure/persistence/almacen.py:16-29`), que expone cuatro operaciones sobre claves lógicas con formato de ruta relativa (`leer_texto`, `escribir_texto`, `eliminar`, `listar`):

- **En modo local (`AlmacenArchivos`, `almacen.py:32-70`):** cada clave `<clave>` corresponde al archivo físico `<ML_DATA_DIR o data>/<clave>` en disco, con escritura atómica mediante archivo temporal `.tmp` y `os.replace`.
- **En modo nube o híbrido (`AlmacenPostgres`, `almacen.py:73-147`):** cada clave `<clave>` corresponde a una fila en la tabla PostgreSQL:

```
CREATE TABLE IF NOT EXISTS documentos (
    clave TEXT PRIMARY KEY,
    contenido TEXT NOT NULL,
    actualizado TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

- `ml_theme`: tema visual (`"dark"` o `"light"`, `frontend/src/App.tsx:86-98`).
- `ml-borradores-revision-guia`: borradores de notas y estados (`entendida`/`con_dudas`) mientras el estudiante revisa una guía offline antes de confirmarla (`frontend/src/lib/guias.ts:74-108`).
- `ml_tutoria_font_size`: tamaño de fuente del lector de tutoría (`frontend/src/views/TutoriaView.tsx:66-70`).

3.4. Qué escribe o consume fuera de `AlmacenDocumentos`

1. **Bóveda de Obsidian (`BovedaObsidianAdapter`, `infrastructure/obsidian/boveda_obsidian.py:18-133`):**
   - Escribe archivos `.md` dentro de la ruta `obsidian_vault` al ingestar materiales (`guardar_nota`), analizar proyectos (`guardar_nota`) o finalizar una tutoría (`guardar_resumen` + actualización de `<materia>.md`).
   - Elimina archivos `.md` al desvincular un material (`eliminar_material`) o vaciar materiales (`vaciar_materiales`), verificando mediante `os.path.realpath` que el archivo esté estrictamente dentro de `self.ruta_boveda` y que su nombre termine en `_Notas.md` o esté bajo la carpeta `Proyectos` (`boveda_obsidian.py:122-133`).
2. **Archivos temporales en el servidor:**
   - Al extraer un sílabo (`apps/api/routers/silabo.py:31-49`) o subir un archivo de material suelto (`apps/api/routers/material.py:102-117`), el endpoint crea un directorio temporal con `tempfile.mkdtemp()`, guarda el archivo subido, lo procesa y lo elimina en el bloque `finally` con `shutil.rmtree(temp_dir, ignore_errors=True)`.
3. **API externa de Google Gemini (`LLM_GeminiAdapter`, `infrastructure/llm/gemini_adapter.py:21-262`):**
   - Consume cuota de la API de Gemini usando la clave configurada en `config/ajustes.json` o en la variable de entorno `GEMINI_API_KEY` (`settings.py:57-73`).
   - Implementa hasta 3 reintentos automáticos con espera progresiva (`15 * (intento + 1)` segundos) cuando recibe errores de cuota (`429` / `RESOURCE_EXHAUSTED`, `gemini_adapter.py:147-153`).

---

4. Casos de uso del sistema

A continuación se especifican los **20 casos de uso (`CU-01` a `CU-20`)** que cubren la totalidad de las vistas web, endpoints HTTP, comandos CLI y scripts del sistema.

CU-01: Gestionar asignaturas del catálogo (listar, crear, renombrar y eliminar)

- **Actor:** Estudiante.
- **Precondiciones:** Servidor `ml-web` en ejecución.
- **Flujo principal:**
  1. **Consultar catálogo:** al abrir la vista «Asignaturas» (`frontend/src/views/AsignaturasView.tsx:41-61`), el frontend invoca `api.obtenerMaterias()` (`frontend/src/api/client.ts:152`) → `GET /api/materias` (`backend/src/modular_learning/apps/api/routers/materias.py:22-29`) → `MateriasService.listar_materias()` y `listar_catalogo()` (`core/application/materias.py:30-36`) → `JsonMateriaRepo.listar()` (`infrastructure/persistence/json_repositorios.py:56-59`). En paralelo consulta cronogramas (`GET /api/materias/cronogramas`) y materiales registrados (`GET /api/material/registrados`) para mostrar cuántas clases y materiales tiene cada asignatura (`AsignaturasView.tsx:45-49, 134-137`).
  2. **Crear asignatura:** el estudiante escribe el nombre en el formulario superior y pulsa «Añadir asignatura» (`AsignaturasView.tsx:68-86`). Se llama a `api.agregarMateria(limpio)` (`client.ts:154-157`) → `POST /api/materias` (`routers/materias.py:32-38`) → `MateriasService.agregar_materia(nombre)` (`core/application/materias.py:42-45`) → `JsonMateriaRepo.agregar_materia(nombre)` (`json_repositorios.py:87-89`), que crea `Materia.desde_nombre(nombre)` y la persiste en `config/materias.json`.
  3. **Renombrar asignatura:** el estudiante pulsa el icono de lápiz junto a una asignatura, edita el nombre y confirma (`AsignaturasView.tsx:88-112`). Se invoca `api.renombrarMateria(slug, limpio)` (`client.ts:159-162`) → `PUT /api/materias/{slug}` (`routers/materias.py:41-51`) → `MateriasService.renombrar_materia(slug, nombre)` (`core/application/materias.py:47-54`), que verifica que el nuevo nombre produzca o reemplace la entrada del `slug` en `JsonMateriaRepo.guardar()` (`json_repositorios.py:65-73`).
  4. **Eliminar asignatura:** el estudiante pulsa el icono de papelera y confirma en el modal (`AsignaturasView.tsx:114-129`). Se invoca `api.eliminarMateria(slug)` (`client.ts:164`) → `DELETE /api/materias/{slug}` (`routers/materias.py:54-62`) → `MateriasService.eliminar_materia(slug, borrar_cronograma=True)` (`core/application/materias.py:56-67`), que ejecuta tres borrados coordinados:
     - Elimina la asignatura de `config/materias.json` (`JsonMateriaRepo.eliminar`, `json_repositorios.py:75-83`).
     - Elimina `schedules/<slug>_cronograma.json` (`JsonCronogramaRepo.eliminar_cronograma`, `json_repositorios.py:186-190`).
     - Elimina todas las entradas de esa asignatura en `config/registro_materiales.json` (`JsonRegistroMaterialesRepo.eliminar_de_materia`, `json_repositorios.py:373-379`).
- **Flujos alternativos y errores:**
  - Si al crear o renombrar el nombre está vacío, `MateriasService` lanza `ValueError` y el router responde `HTTP 400` (`routers/`

materias.py:37, 48).

- Si al renombrar o eliminar el `slug` no existe en `config/materias.json`, el servicio devuelve `None` / `False` y el router responde `HTTP 404` (`routers/materias.py:50, 60`).
- **Discrepancias asociadas: DISC-03 (P14)** (`agregar_materia` sobrescribe el nombre visible si otra operación envía el mismo `slug` con distinta grafía) y **DISC-11** (`eliminar_materia` no borra los proyectos de la asignatura en `config/proyectos.json`).

- **Datos que lee y escribe:**
  - Lee y escribe: `config/materias.json`, `schedules/<slug>_cronograma.json`, `config/registro_materiales.json`.
- **Resultado esperado:** Catálogo de asignaturas actualizado sin duplicados por tildes o mayúsculas; al eliminar una asignatura desaparecen también su cronograma y su registro de materiales.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: vista «Asignaturas» (`frontend/src/views/AsignaturasView.tsx:16-308`) y modal rápido «Nueva asignatura» en «Sílabos y Cronogramas» (`frontend/src/views/SilabosView.tsx:199-219`).
  - Endpoints: `GET /api/materias`, `POST /api/materias`, `PUT /api/materias/{slug}`, `DELETE /api/materias/{slug}` (`backend/src/modular_learning/apps/api/routers/materias.py:22-62`).
  - Servicio y repositorio: `MateriasService` (`core/application/materias.py:22-67`), `JsonMateriaRepo` (`infrastructure/persistence/json_repositorios.py:36-89`).

CU-02: Subir, extraer e importar un sílabo (Web y CLI `ml es`)

- **Actor:** Estudiante.
- **Precondiciones:** Disponer de un archivo de sílabo en formato `.pdf` o `.docx` y tener configurada una clave válida de Gemini.
- **Flujo principal (Interfaz Web):**
  1. En la vista «Sílabos y Cronogramas», el estudiante abre el panel colapsable «Subir archivo de sílabo (.pdf, .docx)» (`frontend/src/views/SilabosView.tsx:954-1035`) y selecciona un archivo (`handleCambioArchivoSubida`, líneas 412-427).
  2. Al pulsar «Extraer con IA» (`handleSubirYExtraer`, `SilabosView.tsx:429-456`), el cliente envía un `FormData` mediante `api.extraerSilabo(subidaArchivo, subidaMateria)` (`frontend/src/api/client.ts:199-205`) → `POST /api/silabo/extraer` (`backend/src/modular_learning/apps/api/routers/silabo.py:23-50`).
  3. El endpoint verifica que la extensión sea `.pdf` o `.docx` (`routers/silabo.py:28-29`), guarda el archivo en un directorio temporal (`líneas 31-35`) y si no se indicó nombre de materia llama a `SilaboService.sugerir_nombre_materia(nombre_limpio)` (`core/application/silabo.py:72-76`).
  4. `SilaboService.extraer_cronograma(ruta_temp, nombre_materia)` (`core/application/silabo.py:97-117`) extrae el texto con `LectorDocumentosLocal.extraer_texto(ruta_temp)` (`infrastructure/documents/lector_documentos.py:40-61`), construye el prompt (`silabo.py:29-55`) y solicita a Gemini un JSON estructurado según el esquema `CronogramaExtraido` (`silabo.py:19-24`) mediante `LLM_GeminiAdapter.generar_json_estructurado` (`infrastructure/llm/gemini_adapter.py:201-227`).
  5. El endpoint devuelve la vista previa (`clases` y `materia_sugerida`) sin guardarla aún en disco (`routers/silabo.py:42-47`). El directorio temporal se borra en el bloque `finally` (`línea 49`).
  6. El estudiante revisa la previsualización en `SilabosView.tsx:1037-1091` y pulsa «Guardar Cronograma» (`handleConfirmarSubida`, líneas 458-484), lo que invoca `api.guardarSilabo(subidaMateria, previewSubida.clases)` (`client.ts:207-212`) → `POST /api/silabo/guardar` (`routers/silabo.py:53-67`).
  7. `SilaboService.guardar_cronograma(materia, clases)` (`core/application/silabo.py:119-125`) persiste el cronograma en `schedules/<slug>_cronograma.json` (`JsonCronogramaRepo.guardar_cronograma`, `json_repositorios.py:167-184`, asignando `id` a cada clase y `estado: "pendiente"`) y registra la asignatura en el catálogo con `JsonMateriaRepo.agregar_materia(materia)`.
- **Flujo alternativo (CLI `ml es`):**
  1. El estudiante ejecuta `ml es` (`backend/src/modular_learning/apps/cli/main.py:28-33` → `apps/cli/commands/es.py:12-80`).
  2. La CLI solicita una ruta (o usa `silabos_dir` por defecto, `es.py:21-25`). Si es una carpeta, lista todos los `.pdf` y `.docx` con `lector_documentos.listar_archivos` (`es.py:34`).
  3. Por cada archivo, sugiere un nombre limpio con `sugerir_nombre_materia`, permite confirmarlo o cambiarlo por consola (`es.py:44-47`), llama a `extraer_cronograma` y luego directamente a `guardar_cronograma` (`es.py:52-66`).
- **Errores:**
  - Extensión distinta de `.pdf`/`.docx` → `HTTP 400` (`routers/silabo.py:29`).
  - Si el archivo no contiene texto extraíble o falla Gemini → `ValueError` en `SilaboService.extraer_cronograma` (`silabo.py:102, 114-116`) → `HTTP 400` (`routers/silabo.py:41`).
  - **Discrepancias asociadas: DISC-01 (P12)**, **DISC-03 (P14)** (guardar el sílabo con un nombre en mayúsculas o sin tildes pisa el nombre del catálogo) y **DISC-04 (P15)** (el campo «Asignatura» al subir un sílabo es un input de texto libre en vez de un selector del catálogo).
- **Datos que lee y escribe:**
  - Lee: archivo `.pdf`/`.docx` subido o desde `silabos_dir`.

- Escribe: `schedules/<slug>_cronograma.json` y `config/materias.json`. Consume cuota de Gemini.

- **Resultado esperado:** Cronograma estructurado guardado para la asignatura y visible inmediatamente en la vista «Sílabos y Cronogramas».
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/SilabosView.tsx:412-484, 954-1091`.
  - CLI: `ml es` (`backend/src/modular_learning/apps/cli/commands/es.py:12-80`).
  - Endpoints: `POST /api/silabo/extraer`, `POST /api/silabo/guardar` (`apps/api/routers/silabo.py:23-67`).
  - Servicio: `SilaboService` (`core/application/silabo.py:61-125`).

CU-03: Editar el cronograma de clases, estados y cortes de parcial

- **Actor:** Estudiante.
- **Precondiciones:** Existir al menos una asignatura en el catálogo.
- **Flujo principal:**
  1. **Cargar cronograma y parciales:** al seleccionar una asignatura en las píldoras superiores de «Sílabos y Cronogramas» (`frontend/src/views/SilabosView.tsx:161-189`), se llama en paralelo a:
     - `api.obtenerCronograma(materia)` (`client.ts:169`) → `GET /api/materias/{materia}/cronograma` (`apps/api/routers/materias.py:72-78`) → `MateriasService.obtener_cronograma` (`core/application/materias.py:70-73`) → `JsonCronogramaRepo.obtener_cronograma` (`json_repositorios.py:151-165`).
     - `api.obtenerParciales(materia)` (`client.ts:190`) → `GET /api/materias/{materia}/parciales` (`routers/materias.py:154-159`) → `MateriasService.obtener_parciales` (`core/application/materias.py:146-150`) → `JsonCronogramaRepo.obtener_parciales` (`json_repositorios.py:293-301`).
  2. **Añadir una clase manualmente:** el estudiante pulsa «Añadir clase», completa `tema`, `fecha`, `objetivo` y `contenido` (separado por comas) y guarda (`SilabosView.tsx:314-354`). Se invoca `api.agregarClase(materiaActiva, payload)` (`client.ts:173-177`) → `POST /api/materias/{materia}/clases` (`routers/materias.py:100-110`) → `MateriasService.agregar_clase` (`core/application/materias.py:96-108`), que genera un `nuevo_id_clase()`, fija `estado: "pendiente"`, añade la clase al cronograma y asegura que la materia exista en `JsonMateriaRepo`.
  3. **Editar una clase existente:** el estudiante pulsa el icono de editar en una fila del cronograma, modifica los campos y guarda (`SilabosView.tsx:266-312`). Se invoca `api.editarClase(materiaActiva, editandoId, payload)` (`client.ts:179-183`) → `PUT /api/materias/{materia}/clases/{clase_id}` (`routers/materias.py:113-126`) → `MateriasService.editar_clase` (`core/application/materias.py:110-124`), que localiza la clase por `clase["id"] == clase_id` y actualiza sus campos.
  4. **Alternar estado (`pendiente` / `completado`):** el estudiante hace clic en el círculo de estado de una clase (`handleToggleEstado`, `SilabosView.tsx:383-409`). Se invoca `api.cambiarEstadoClase(materiaActiva, clase.id, nuevoEstado)` (`client.ts:186-189`) → `PATCH /api/materias/{materia}/clases/{clase_id}/estado` (`routers/materias.py:141-151`) → `MateriasService.cambiar_estado_clase` (`core/application/materias.py:139-143`) → `JsonCronogramaRepo.marcar_clase` (`json_repositorios.py:227-259`). Si pasa a `"completado"`, graba `fecha_completado` con la fecha actual; si vuelve a `"pendiente"`, elimina `fecha_completado`.
  5. **Eliminar una clase individual:** el estudiante pulsa el icono de papelera en la fila de la clase y confirma (`SilabosView.tsx:356-381`). Se invoca `api.eliminarClase(materiaActiva, id)` (`client.ts:184-185`) → `DELETE /api/materias/{materia}/clases/{clase_id}` (`routers/materias.py:129-138`) → `MateriasService.eliminar_clase` (`core/application/materias.py:126-137`).
  6. **Definir o quitar cortes de parcial:** en la sección «Cortes de Parcial» (`SilabosView.tsx:664-738`), el estudiante añade o quita cortes y el cliente invoca `api.guardarParciales(materiaActiva, nuevos)` (`client.ts:192-196`) → `POST /api/materias/{materia}/parciales` (`routers/materias.py:162-172`) → `MateriasService.guardar_parciales` (`core/application/materias.py:152-159`) → `JsonCronogramaRepo.guardar_parciales` (`json_repositorios.py:303-315`).
  7. **Vista global de calendario y próximas clases:**
     - En la sub-pestaña «Calendario» de «Sílabos y Cronogramas» (`SilabosView.tsx:557-572` → `frontend/src/views/CalendarioView.tsx:43-295`), se invoca `api.obtenerCronogramas()` (`client.ts:166`) → `GET /api/materias/cronogramas` (`routers/materias.py:65-69`) para mostrar una línea de tiempo consolidada de todas las asignaturas con filtros por estado y asignatura.
     - En `TutoriaView.tsx:130` y `QuizView.tsx:100`, se invoca `api.obtenerClasesPendientes(materia)` (`client.ts:171`) → `GET /api/materias/{materia}/clases` (`routers/materias.py:92-97`) → `MateriasService.obtener_proximas_clases(materia, limite=20)` (`core/application/materias.py:75-79`) para poblar los selectores de clases pendientes.
- **Flujos alternativos y errores:**
  - Si `clase_id` no existe al editar, borrar o cambiar estado → `HTTP 404` (`routers/materias.py:123, 136, 149`).
  - Si el estado enviado en `PATCH` no es `"pendiente"` ni `"completado"` → `ValueError` en `MateriasService.cambiar_estado_clase` (`materias.py:140-141`) → `HTTP 400`.
  - **Discrepancia asociada: DISC-10** (en `SilabosView.tsx:666, 708-711` el corte de parcial guarda una fecha en texto libre dentro de `hasta`, mientras que `GuiaView.tsx:346, 360` espera que `hasta` contenga el `id` de la clase límite).

- **Datos que lee y escribe:**
  - Lee y escribe: `schedules/<slug>_cronograma.json` y `config/materias.json`.
- **Resultado esperado:** Clases añadidas, editadas, marcadas o eliminadas inmediatamente en el cronograma de la asignatura.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/SilabosView.tsx` y `frontend/src/views/CalendarioView.tsx`.
  - Endpoints: `GET /api/materias/cronogramas`, `GET /api/materias/{materia}/cronograma`, `GET /api/materias/{materia}/clases`, `POST /api/materias/{materia}/clases`, `PUT /api/materias/{materia}/clases/{clase_id}`, `DELETE /api/materias/{materia}/clases/{clase_id}`, `PATCH /api/materias/{materia}/clases/{clase_id}/estado`, `GET /api/materias/{materia}/parciales`, `POST /api/materias/{materia}/parciales` (`apps/api/routers/materias.py:65-172`).

CU-04: Vaciar el cronograma de una asignatura

- **Actor:** Estudiante.
- **Precondiciones:** Tener seleccionada una asignatura con clases cargadas en la vista «Sílabos y Cronogramas».
- **Flujo principal:**
  1. El estudiante pulsa el botón «Vaciar cronograma» en la cabecera de la asignatura activa (`frontend/src/views/SilabosView.tsx:766-773`).
  2. Se abre un diálogo de confirmación que indica: *«¿Vaciar el cronograma de "{materiaActiva}"? Se borrarán todas sus clases pero se conservará la asignatura en el catálogo.»* (`SilabosView.tsx:247-249`).
  3. Al confirmar, el frontend llama a `api.eliminarCronograma(materiaActiva)` (`frontend/src/api/client.ts:167`) → `DELETE /api/materias/{materia}/cronograma` (`backend/src/modular_learning/apps/api/routers/materias.py:81-89`).
  4. El endpoint invoca `servicio.eliminar_cronograma(materia, borrar_materia=True)` (`routers/materias.py:86`) → `MateriasService.eliminar_cronograma` (`core/application/materias.py:87-94`).
- **Flujos alternativos y errores:**
  - Si no existía archivo de cronograma ni materia en el catálogo → `HTTP 404` (`routers/materias.py:88`).
  - **Discrepancia crítica asociada — DISC-02 (P13):** A pesar de lo que promete el diálogo en `SilabosView.tsx:248` y la especificación funcional, el endpoint pasa `borrar_materia=True` por defecto (`routers/materias.py:86` y `core/application/materias.py:87`), lo que ejecuta `self.eliminar_materia(materia, borrar_cronograma=False)` (`materias.py:92-93`). Como consecuencia, **vaciar el cronograma borra también la asignatura de `config/materias.json` y borra sus materiales registrados de `config/registro_materiales.json`**. Además, `SilabosView.tsx:255-257` no recarga el catálogo tras el borrado, dejando seleccionada en pantalla una asignatura fantasma.
- **Datos que lee y escribe:**
  - Elimina: `schedules/<slug>_cronograma.json`, y debido a **DISC-02 (P13)** modifica también `config/materias.json` y `config/registro_materiales.json`.
- **Resultado esperado (según especificación):** El cronograma queda sin clases y la asignatura permanece en el catálogo.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/SilabosView.tsx:245-264, 766-773`.
  - Endpoint: `DELETE /api/materias/{materia}/cronograma` (`apps/api/routers/materias.py:81-89`).
  - Servicio: `MateriasService.eliminar_cronograma` (`core/application/materias.py:87-94`).

CU-05: Escanear carpetas locales e ingestar material pendiente en lote (Web SSE y CLI `ml im`)

- **Actor:** Estudiante (en modo local o híbrido).
- **Precondiciones:** Tener configurada la ruta `material_dir` con subcarpetas por asignatura y la ruta `obsidian_vault`.
- **Flujo principal (Interfaz Web):**
  1. **Escaneo:** en la vista «Material de Estudio» (`frontend/src/views/MaterialView.tsx:559-626`) o desde el panel lateral «Material nuevo» en la vista «Hoy» (`frontend/src/views/HoyView.tsx:111-114, 321-356`), el estudiante pulsa «Revisar carpetas» (`handleEscanear`, `MaterialView.tsx:314-328`). Se invoca `api.escanearMaterial(materiaEscaneo)` (`client.ts:216-222`) → `GET /api/material/escanear` (`apps/api/routers/material.py:54-61`) → `IngestaService.escanear()` o `escanear_materia()` (`core/application/ingesta.py:205-253, 279-280`).
  2. `IngestaService.escanear_materia` busca la subcarpeta de la asignatura dentro de `material_dir` (`_resolver_carpeta_materia`, `ingesta.py:241-253`), excluye cualquier archivo dentro de subcarpetas `Proyectos` (que delega a `ProyectosService.escanear_proyectos_en_carpeta`, líneas 211-220), clasifica los archivos `.pdf`, `.docx`, `.txt` y `.md` en `pendientes` o `ingestados` consultando `JsonRegistroMaterialesRepo.verificar_material_ingestado` (`json_repositorios.py:342-344`), y cuenta los `.pptx` en `no_soportados` (`ingesta.py:224-226`).
  3. **Sincronización en streaming (SSE):** cuando hay archivos pendientes, el estudiante pulsa «Procesar N pendientes» (`handleSincronizar`, `MaterialView.tsx:330-387` o `HoyView.tsx:175-186`). El cliente abre un stream `POST /api/material/sincronizar/stream` (`client.ts:224-264` → `apps/api/routers/material.py:64-84`) → `IngestaService.ingestar_pendientes_stream` (`core/application/ingesta.py:282-323`).

4. Por cada archivo pendiente, el generador emite un evento SSE `{"tipo": "procesando", ...}` y ejecuta `IngestaService.ingestar_archivo` (`ingesta.py:104-169`):
   - Clasifica el archivo por subcarpeta (`_clasificar_archivo`, líneas 55-68).
   - Si es `.pdf`, llama a `LLM_GeminiAdapter.procesar_documento_multimodal` (`gemini_adapter.py:165-199`); si es `.docx`, `.txt` o `.md`, extrae el texto plano con `LectorDocumentosLocal.extraer_texto` y llama a `LLM_GeminiAdapter.generar_texto` (`ingesta.py:75-84`).
   - Guarda la nota Markdown resultante en `<obsidian_vault>/<materia>/<SubcarpetaTipo>/<base>_Notas.md` (`BovedaObsidianAdapter.guardar_nota`, `infrastructure/obsidian/boveda_obsidian.py:28-42`).
   - Registra el archivo en `config/registro_materiales.json` (`JsonRegistroMaterialesRepo.registrar_material`, `json_repositorios.py:346-359`).
   - Extrae metadatos con IA (`_extraer_metadatos`, `ingesta.py:86-102`) y alinea el material con la clase correspondiente del cronograma (`asignar_material_a_clase`, `core/domain/alineador_curricular.py:51-109`).
   - Si es una `actividad` y existe un plugin activo para la asignatura, añade el resumen a `plugins/<id>_context.md` (`ingesta.py:157-167`).
5. Al terminar el lote, emite `{"tipo": "completado", "procesados": N, "errores": [...]}` (`ingesta.py:317-323`).

- **Flujo alternativo (CLI `ml im`):**
  1. El estudiante ejecuta `ml im` (`apps/cli/main.py:36-41` → `apps/cli/commands/im.py:12-103`).
  2. La CLI lista las asignaturas de `MateriasService.listar_materias()`, pide elegir una, escanea con `ingesta_service.escanear_materia(materia)` (`im.py:37`), muestra los archivos pendientes y los proyectos detectados, y al confirmar procesa cada archivo con `ingesta_service.ingestar_archivo` (`im.py:72`) y cada proyecto nuevo o modificado con `proyectos_service.analizar_e_ingestar_proyecto` (`im.py:89`).
- **Errores y discrepancias asociadas:**
  - **DISC-05 (P16):** Los archivos `.pptx` se omiten en el escaneo como `no_soportados`.
  - **DISC-07:** Los archivos `.docx` solo extraen texto de párrafos (`lector_documentos.py:51-55`), ignorando tablas e imágenes embebidas.
  - **DISC-08:** `HoyView.tsx:111-114` llama a `escanearMaterial()` incluso cuando `en_nube` es `true`, y `MaterialView.tsx:559-626` sigue mostrando el escáner de carpetas en modo nube.
- **Datos que lee y escribe:**
  - Lee: archivos en `material_dir`, `config/registro_materiales.json`, `schedules/<slug>_cronograma.json`, `plugins/<id>_context.md`.
  - Escribe: notas `.md` en `obsidian_vault`, `config/registro_materiales.json`, `schedules/<slug>_cronograma.json` y `plugins/<id>_context.md`.
- **Resultado esperado:** Todos los materiales pendientes quedan convertidos en notas de Obsidian, registrados y vinculados a su clase en el cronograma.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/MaterialView.tsx:314-387, 559-626, 707-741` y `frontend/src/views/HoyView.tsx:111-114, 175-186, 321-356`.
  - CLI: `ml im` (`backend/src/modular_learning/apps/cli/commands/im.py:12-103`).
  - Endpoints: `GET /api/material/escanear`, `POST /api/material/sincronizar/stream` (`apps/api/routers/material.py:54-84`).
  - Servicio: `IngestaService` (`core/application/ingesta.py:104-323`).

CU-06: Subir e ingestar un archivo de material suelto

- **Actor:** Estudiante (en modo local, nube o híbrido).
- **Precondiciones:** Tener un archivo `.pdf`, `.docx`, `.txt` o `.md` en el dispositivo desde el que se abre el navegador.
- **Flujo principal:**
  1. En la vista «Material de Estudio», dentro de la tarjeta «Subir un archivo suelto» (`frontend/src/views/MaterialView.tsx:744-842`), el estudiante escribe el nombre de la asignatura y selecciona o arrastra un archivo (`handleCambioArchivoSubida`, líneas 389-400).
  2. Al pulsar «Procesar con IA» (`handleSubirArchivo`, `MaterialView.tsx:402-436`), el cliente invoca `api.ingestarMaterialArchivo(subidaArchivo, subidaMateria)` (`frontend/src/api/client.ts:266-272`) → `POST /api/material/ingestar` (`backend/src/modular_learning/apps/api/routers/material.py:87-118`).
  3. El endpoint comprueba la extensión:
     - Si está en `EXTENSIONES_NO_SOPORTADAS_WEB` (`.pptx`), responde `HTTP 400` con un mensaje explicativo (`routers/material.py:97-101`).
     - Si no está en `EXTENSIONES_MATERIAL` (`.pdf`, `.docx`, `.txt`, `.md`), responde `HTTP 400` (`routers/material.py:102-105`).
  4. El endpoint llama a `MateriasService.agregar_materia(materia)` (`routers/material.py:106`), guarda el archivo en un

directorio temporal (líneas 107-111) y ejecuta `IngestaService.ingestar_archivo(ruta_temp, materia, forzar=True)` (línea 112). Al estar en una carpeta temporal sin nombre de categoría, `_clasificar_archivo` lo clasifica siempre como `lectura` (ingesta.py:68).

5. En el bloque `finally`, borra el directorio temporal (routers/material.py:117) y devuelve la ruta de la nota generada y la clase asociada (líneas 113-115).

- **Flujos alternativos y errores:**
  - Archivo `.pptx` u otra extensión no admitida → `HTTP 400`.
  - **Discrepancias asociadas: DISC-03 (P14)** (`agregar_materia` y `ingestar_archivo` usan el texto literal enviado por el formulario sin resolverlo contra el catálogo, pudiendo sobrescribir el nombre en `config/materias.json` y crear otra carpeta en Obsidian), **DISC-04 (P15)** (el campo «Asignatura» es un `<Input>` de texto libre en `MaterialView.tsx:758-764`), **DISC-05 (P16)** (bloqueo de `.pptx`) y **DISC-06 (P17)** (el formulario de subida suelta está siempre desplegado por encima del historial de materiales ingestados).
- **Datos que lee y escribe:**
  - Lee: archivo subido por multipart, `schedules/<slug>_cronograma.json`, `plugins/<id>_context.md`.
  - Escribe: `config/materias.json`, nota `.md` en `obsidian_vault/<materia>/Lecturas/`, `config/registro_materiales.json` y `schedules/<slug>_cronograma.json`.
- **Resultado esperado:** El archivo suelto queda procesado en Obsidian, registrado en el historial y alineado con su clase del cronograma.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/MaterialView.tsx:389-436, 744-842`.
  - Endpoint: `POST /api/material/ingestar` (apps/api/routers/material.py:87-118).
  - Servicio: `IngestaService.ingestar_archivo` (core/application/ingesta.py:104-169).

CU-07: Revisar historial de materiales ingestados y desvincular un material

- **Actor:** Estudiante.
- **Precondiciones:** Existir materiales previamente ingestados en `config/registro_materiales.json`.
- **Flujo principal:**
  1. Al abrir «Material de Estudio», `cargarMateriales()` (frontend/src/views/MaterialView.tsx:274-283) llama a `api.obtenerMaterialesRegistrados()` (client.ts:214) → `GET /api/material/registrados` (apps/api/routers/material.py:26-30) → `IngestaService.listar_materiales_ingestados()` (core/application/ingesta.py:171-173) → `JsonRegistroMaterialesRepo.listar_materiales()` (json_repositorios.py:361-362).
  2. La tabla «Materiales Ingestados» (`MaterialView.tsx:845-920`) permite filtrar por asignatura y muestra para cada registro su nombre de archivo, asignatura, tipo y fecha de ingesta.
  3. Cuando el estudiante pulsa el botón de papelera junto a un material y confirma en el diálogo (`handleEliminarMaterial`, `MaterialView.tsx:438-460`), se invoca `api.eliminarMaterial(nombre)` (client.ts:276-279) → `DELETE /api/material/{nombre_archivo:path}` (apps/api/routers/material.py:42-51) → `IngestaService.eliminar_material(nombre_archivo)` (core/application/ingesta.py:175-203).
  4. `IngestaService.eliminar_material` realiza tres acciones:
     - Elimina la entrada de `config/registro_materiales.json` (`JsonRegistroMaterialesRepo.eliminar_material`, json_repositorios.py:364-371).
     - Si tenía `ruta_obsidian`, elimina la nota `.md` de la bóveda (`BovedaObsidianAdapter.eliminar_nota`, boveda_obsidian.py:122-133).
     - Si tenía `materia` y existe cronograma, quita el archivo de `material_transversal` y de `materiales_asociados` de todas las clases del cronograma (ingesta.py:189-201).
- **Flujos alternativos y errores:**
  - Si `nombre_archivo` no existe en `config/registro_materiales.json` → `HTTP 404` (routers/material.py:49).
  - **Discrepancia asociada: DISC-06 (P17)** (la tabla de materiales ingestados se muestra al final de la vista en vez de ser la sección principal).
- **Datos que lee y escribe:**
  - Lee y escribe: `config/registro_materiales.json`, `schedules/<slug>_cronograma.json`, y elimina la nota correspondiente en `obsidian_vault`.
- **Resultado esperado:** El material desaparece del historial, se borra su nota de Obsidian y se desvincula de las clases del cronograma.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/MaterialView.tsx:274-283, 438-460, 845-920`.
  - Endpoints: `GET /api/material/registrados`, `DELETE /api/material/{nombre_archivo:path}` (apps/api/routers/material.py:26-30, 42-51).

- Servicio: `IngestaService.listar_materiales_ingestados` y `eliminar_material` (`core/application/ingesta.py:171-203`).

CU-08: Escanear, ingestar, verificar cambios, actualizar y desvincular proyectos de asignatura

- **Actor:** Estudiante.
- **Precondiciones:** En modo local/híbrido, contar con carpetas de proyecto dentro de `<material_dir>/<Asignatura>/Proyectos/...`.
- **Flujo principal:**
  1. **Listar y escanear proyectos:** al cargar «Material de Estudio», `cargarProyectos()` (`frontend/src/views/MaterialView.tsx:285-312`) invoca `api.listarProyectos()` (`client.ts:282`) → `GET /api/proyectos` (`apps/api/routers/proyectos.py:23-29`) y `api.escanearProyectos()` (`client.ts:285-290`) → `GET /api/proyectos/escanear` (`routers/proyectos.py:32-48`).
  2. `GET /api/proyectos/escanear` recorre las asignaturas, llama a `ProyectosService.escanear_proyectos_en_carpeta(ruta, nombre)` (`core/application/proyectos.py:58-66`) → `InspectorProyectosLocal.escanear` (`infrastructure/proyectos/inspector_local.py:83-99`) y devuelve cada carpeta detectada indicando si ya está `registrado` o si tiene `cambios` (`ProyectosService.verificar_estado_proyecto`, `proyectos.py:161-186`).
  3. **Ingestar un proyecto detectado:** en la sección «Proyectos de Asignatura» (`MaterialView.tsx:629-702`), el estudiante pulsa «Analizar con IA» sobre un proyecto nuevo (`handleIngestarProyecto`, líneas 483-502), llamando a `api.ingestarProyecto(ruta_origen, materia, parcial)` (`client.ts:297-302`) → `POST /api/proyectos/ingestar` (`routers/proyectos.py:73-88`) → `ProyectosService.analizar_e_ingestar_proyecto` (`core/application/proyectos.py:118-159`):
     - Calcula `HuellaProyecto` (`inspector_local.py:101-127`), detecta el tipo (`software`, `iot`, `documental_auditoria`, `documento_rector`) y extrae el contexto técnico de los archivos del proyecto (`inspector_local.py:151-183`).
     - Genera una ficha técnica en Markdown con Gemini usando un prompt especializado según el tipo de proyecto (`_construir_prompt_proyecto`, `proyectos.py:68-116`).
     - Guarda la ficha en `<obsidian_vault>/<materia>/Proyectos/<nombre>.md` (`proyectos.py:141-145`).
     - Persiste el objeto `Proyecto` en `config/proyectos.json` (`JsonProyectosRepo.guardar`, `json_repositorios.py:421-426`) y añade también una entrada `[PROYECTO] {nombre} ({parcial})` en `config/registro_materiales.json` (`proyectos.py:154-158`).
  4. **Verificar estado y actualizar un proyecto modificado:** si `verificar_estado_proyecto` (`GET /api/proyectos/{id_proyecto:path}/estado`, `routers/proyectos.py:51-59`) detecta cambios en los archivos del proyecto, la tarjeta muestra el botón «Actualizar ficha» (`handleActualizarProyecto`, `MaterialView.tsx:504-521`), que llama a `api.actualizarProyecto(id)` (`client.ts:304-308`) → `POST /api/proyectos/{id_proyecto:path}/actualizar` (`routers/proyectos.py:91-102`) → `ProyectosService.actualizar_proyecto` (`proyectos.py:188-199`), re-ejecutando el análisis y actualizando la nota y la huella. También existe `GET /api/proyectos/{id_proyecto:path}` (`routers/proyectos.py:62-70`) para consultar el detalle de un proyecto por su `id`.
  5. **Desvincular un proyecto:** el estudiante pulsa el icono de papelera en un proyecto registrado (`handleEliminarProyecto`, `MaterialView.tsx:523-536`), invocando `api.eliminarProyecto(id)` (`client.ts:310-313`) → `DELETE /api/proyectos/{id_proyecto:path}` (`routers/proyectos.py:105-113`) → `ProyectosService.eliminar_proyecto` (`proyectos.py:55-56`) → `JsonProyectosRepo.eliminar` (`json_repositorios.py:444-450`).
- **Flujos alternativos y errores:**
  - Proyecto inexistente o carpeta borrada al actualizar → `HTTP 400` / `HTTP 404` (`routers/proyectos.py:68, 86, 100, 111`).
  - **Discrepancia asociada — DISC-11:** `ProyectosService.eliminar_proyecto` borra el proyecto de `config/proyectos.json`, pero **no elimina** su registro `[PROYECTO] ...` de `config/registro_materiales.json` ni borra la ficha `.md` en Obsidian. Además, las fichas de `ProyectosService` no se inyectan en los prompts del tutor (`TutoriaService` solo lee `PluginsService.obtener_contexto_plugin`).
- **Datos que lee y escribe:**
  - Lee: carpetas bajo `<material_dir>/<Asignatura>/Proyectos`, `config/proyectos.json`.
  - Escribe: `config/proyectos.json`, `config/registro_materiales.json`, `<obsidian_vault>/<materia>/Proyectos/<nombre>.md`.
- **Resultado esperado:** Proyectos analizados como fichas técnicas en Obsidian, con detección de cambios por huella de archivos.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/MaterialView.tsx:285-312, 483-536, 629-702`.
  - CLI: integrado dentro de `ml im` (`apps/cli/commands/im.py:29-31, 51-57, 81-94`).
  - Endpoints: `GET /api/proyectos`, `GET /api/proyectos/escanear`, `GET /api/proyectos/{id_proyecto:path}/estado`, `GET /api/proyectos/{id_proyecto:path}`, `POST /api/proyectos/ingestar`, `POST /api/proyectos/{id_proyecto:path}/actualizar`, `DELETE /api/proyectos/{id_proyecto:path}` (`apps/api/routers/proyectos.py:23-113`).
  - Servicio e inspector: `ProyectosService` (`core/application/proyectos.py:25-207`), `InspectorProyectosLocal` (`infrastructure/proyectos/inspector_local.py:44-183`).

CU-09: Vaciar todos los materiales y proyectos ingestados

- **Actor:** Estudiante.
- **Precondiciones:** Existir materiales registrados en `config/registro_materiales.json` o proyectos en `config/proyectos.json`.
- **Flujo principal:**
  1. En la cabecera de la tabla «Materiales Ingestados» (`frontend/src/views/MaterialView.tsx:862-871`), el estudiante pulsa «Vaciar todo» y confirma en el diálogo (`handleVaciarMateriales`, líneas 462-481).
  2. El cliente invoca `api.vaciarMateriales()` (`frontend/src/api/client.ts:274`) → `POST /api/material/vaciar` (`backend/src/modular_learning/apps/api/routers/material.py:33-39`).
  3. El endpoint ejecuta en secuencia:
     - `IngestaService.vaciar_materiales()` (`core/application/ingesta.py:255-277`): recorre `config/registro_materiales.json`, elimina cada nota `.md` en Obsidian mediante `BovedaObsidianAdapter.eliminar_nota`, y vacía `config/registro_materiales.json` (`JsonRegistroMaterialesRepo.vaciar`, `json_repositorios.py:381-385`).
     - `ProyectosService.vaciar_proyectos()` (`core/application/proyectos.py:201-207`): elimina las fichas de proyectos en Obsidian y vacía `config/proyectos.json` (`JsonProyectosRepo.vaciar`, `json_repositorios.py:452-456`).
- **Flujos alternativos y errores:**
  - Si alguna nota ya no existe en disco, se ignora el error y se contabilizan únicamente las notas efectivamente borradas (`ingesta.py:267-270`).
  - **Discrepancia asociada — DISC-11:** `vaciar_materiales()` borra `config/registro_materiales.json` y las notas de Obsidian, pero **no limpia** las listas `materiales_asociados`, `subtemas_adicionales` ni `material_transversal` que habían sido inyectadas dentro de los cronogramas (`schedules/<slug>_cronograma.json`).
- **Datos que lee y escribe:**
  - Escribe: vacía `config/registro_materiales.json` y `config/proyectos.json`; elimina las notas `.md` generadas en `obsidian_vault`.
- **Resultado esperado:** Historial de materiales y proyectos completamente vacío, permitiendo re-ingestar desde cero.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/MaterialView.tsx:462-481, 862-871` y también invocable desde el asistente de reinicio en `ConfigView.tsx:165`.
  - Endpoint: `POST /api/material/vaciar` (`apps/api/routers/material.py:33-39`).
  - Servicios: `IngestaService.vaciar_materiales` (`core/application/ingesta.py:255-277`) y `ProyectosService.vaciar_proyectos` (`core/application/proyectos.py:201-207`).

CU-10: Realizar una sesión de tutoría interactiva (Web y CLI `ml init`)

- **Actor:** Estudiante.
- **Precondiciones:** Tener una asignatura (y opcionalmente clases en su cronograma) y clave de Gemini activa.
- **Flujo principal (Interfaz Web):**
  1. **Configurar e iniciar sesión:** en la vista «Sesión de Estudio» (`frontend/src/views/TutoriaView.tsx:100-217`, a la que también se puede llegar pre-rellenada pulsando «Estudiar con tutor» desde `HoyView.tsx:249-256` o al fallar un quiz en `QuizView.tsx:386-399`), el estudiante elige la asignatura, selecciona una clase pendiente del cronograma (o activa el modo «Tema libre»), añade subtemas adicionales o un enfoque particular y pulsa «Iniciar Tutoría» (`handleIniciar`, `TutoriaView.tsx:163-196`).
  2. El cliente invoca `api.iniciarTutoria(materia, tema, extra, enfoque, claseId)` (`frontend/src/api/client.ts:315-327`) → `POST /api/tutoria/iniciar` (`backend/src/modular_learning/apps/api/routers/tutoria.py:33-69`).
  3. El endpoint registra la materia con `MateriasService.agregar_materia(req.materia)` (`routers/tutoria.py:55`) e invoca `TutoriaService.iniciar_sesion` (`core/application/tutoria.py:115-165`):
     - Busca la clase en el cronograma por `clase_id` (o por nombre de `tema` si no se envió `clase_id`) usando `buscar_clase` (`core/domain/cronograma.py:45-66`) para recuperar su `objetivo` y `contenido` oficial.
     - Si el tema tiene menos de 2 subtemas, pide a Gemini que desglose el tema en una ruta de aprendizaje (`_desglosar_tema_con_ia`, `tutoria.py:85-113`).
     - Obtiene la memoria técnica del plugin activo para esa asignatura (`PluginsService.obtener_contexto_plugin`, `plugins.py:67-81`).
     - Devuelve el diccionario `sesion` con `ruta_aprendizaje` (lista de pasos), `objetivo`, `enfoque`, `contexto_plugin` y `clase_id`.
  4. El router guarda la sesión en RAM mediante `SessionStore.guardar_sesion` (`apps/api/session_store.py:17-18`) y devuelve `sesion_id` y `ruta_aprendizaje` (`routers/tutoria.py:58-65`).
  5. **Explicación de cada paso en streaming (SSE):** automáticamente al iniciar o al cambiar de paso (`cargarPaso`, `TutoriaView.tsx:219-280`), el frontend abre un stream `GET /api/tutoria/{sesion_id}/paso/{indice}/stream` (`client.ts:329-370` → `routers/tutoria.py:72-98`). Si el paso no estaba en caché dentro de `sesion["explicaciones_cache"]`, `TutoriaService.explicar_paso_stream` (`tutoria.py:225-238`) construye el prompt (`construir_prompt_explicacion`, `tutoria.py:167-212`) e invoca `LLM_GeminiAdapter.generar_respuesta_stream`

(gemini_adapter.py:229-262), enviando fragmentos JSON {"chunk": "..."} por SSE y guardando al final la explicación completa en caché (routers/tutoria.py:88-89).

6. **Resolver dudas del paso actual:** el estudiante escribe una pregunta en el chat lateral de dudas (handleEnviarDuda, TutoriaView.tsx:282-313), que llama a api.enviarDuda(sesionId, indicePaso, duda) (client.ts:372-378) → POST /api/tutoria/{sesion_id}/duda (routers/tutoria.py:101-115) → TutoriaService.responder_duda (tutoria.py:240-253). La respuesta se devuelve y se anexa a sesion["historial_dudas"].
7. **Finalizar sesión y guardar en Obsidian:** al terminar los pasos, el estudiante pulsa «Finalizar y Guardar en Obsidian» (handleFinalizar, TutoriaView.tsx:315-340), llamando a api.finalizarTutoria(sesionId) (client.ts:380-383) → POST /api/tutoria/{sesion_id}/finalizar (routers/tutoria.py:118-130) → TutoriaService.finalizar_sesion (tutoria.py:255-281), que:
   - Genera una conclusión ejecutiva con Gemini (tutoria.py:263-267).
   - Escribe el resumen completo (explicaciones, dudas resueltas y conclusión) en <obsidian_vault>/<materia>/<tema>.md y actualiza el índice <obsidian_vault>/<materia>.md (BovedaObsidianAdapter.guardar_resumen, boveda_obsidian.py:44-94).
   - Marca la clase como "completado" en el cronograma por clase_id (o por tema si fue tema libre) mediante JsonCronogramaRepo.marcar_clase_completada (json_repositorios.py:261-264).
   - Registra el evento de estudio y crea tarjetas de repaso para los pasos vistos mediante RepasoService.registrar_tutoria (core/application/repaso.py:89-102).
   - Elimina la sesión de SessionStore (routers/tutoria.py:124).

- **Flujo alternativo (CLI `ml init`):**
  1. El estudiante ejecuta ml init (apps/cli/main.py:20-25 → apps/cli/commands/init.py:13-91).
  2. pedir_datos_inicio (apps/cli/ui/interaccion.py:32-104) pide seleccionar materia, elegir una clase pendiente del cronograma o tema libre, y añadir subtemas o enfoque.
  3. Por cada paso de ruta_aprendizaje, transmite la explicación en consola con paneles Rich.Live (init.py:48-55) y abre un bucle interactivo de dudas (init.py:61-74) antes de pasar al siguiente subtema y finalizar con tutoria_service.finalizar_sesion (init.py:82).
- **Errores y discrepancias asociadas:**
  - Si sesion_id no existe en SessionStore (p. ej. tras reiniciar el servidor) → HTTP 404 (routers/tutoria.py:76, 104, 121).
  - **DISC-03 (P14):** Si materia se envía con otra grafía, POST /api/tutoria/iniciar sobrescribe el nombre en config/materias.json y finalizar_sesion escribe la nota de Obsidian en una carpeta con esa grafía.
  - **DISC-09:** En la CLI (ml init), pedir_datos_inicio (apps/cli/ui/interaccion.py:81-86, 104) descarta el id de la clase seleccionada y apps/cli/commands/init.py:29-34 llama a iniciar_sesion sin clase_id, por lo que al cerrar la tutoría desde la CLI se marca la primera clase cuyo nombre de tema coincida (cronograma.py:55-65).
- **Datos que lee y escribe:**
  - Lee: schedules/<slug>_cronograma.json, plugins/<id>_context.md, SessionStore.
  - Escribe: <obsidian_vault>/<materia>/<tema>.md, <obsidian_vault>/<materia>.md, schedules/<slug>_cronograma.json, repaso/tarjetas.json, repaso/historial.json.
- **Resultado esperado:** Sesión completada, resumen guardado en Obsidian, clase marcada como completada y subtemas incorporados al sistema de repaso espaciado.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: frontend/src/views/TutoriaView.tsx:41-594.
  - CLI: ml init (backend/src/modular_learning/apps/cli/commands/init.py:13-91, apps/cli/ui/interaccion.py:32-104).
  - Endpoints: POST /api/tutoria/iniciar, GET /api/tutoria/{sesion_id}/paso/{indice}/stream, POST /api/tutoria/{sesion_id}/duda, POST /api/tutoria/{sesion_id}/finalizar (apps/api/routers/tutoria.py:33-130).
  - Servicio: TutoriaService (core/application/tutoria.py:73-281).

CU-11: Generar y evaluar un quiz de recuerdo activo con IA (Web y CLI `ml quiz`)

- **Actor:** Estudiante.
- **Precondiciones:** Tener al menos una asignatura (y opcionalmente apuntes en Obsidian o clases en el cronograma) y clave de Gemini activa.
- **Flujo principal (Interfaz Web):**
  1. En la vista «Repaso y Evaluación» (frontend/src/views/QuizView.tsx:41-557), el estudiante selecciona el modo «Quiz nuevo», elige la asignatura y opcionalmente escribe o selecciona un tema y el número de preguntas (cantidad, entre 1 y 10, por defecto 3), y pulsa «Generar Quiz» (handleGenerar, QuizView.tsx:108-137).
  2. El cliente invoca api.generarQuiz(materia, tema, cantidad) (frontend/src/api/client.ts:385-395) → GET /api/quiz/generar (backend/src/modular_learning/apps/api/routers/quiz.py:31-42) → EvaluacionService.generar_quiz(materia, tema, cantidad) (core/application/evaluacion.py:92-156):
     - Reúne contexto desde tres fuentes: apuntes existentes en la bóveda de Obsidian

(`BovedaObsidianAdapter.buscar_notas_contexto`, `boveda_obsidian.py:96-119`), memoria técnica del plugin activo (`PluginsService.obtener_contexto_plugin`) y lista de clases completadas en el cronograma (`JsonCronogramaRepo.obtener_temas_completados`, `json_repositorios.py:216-225`).
    - Solicita a Gemini un JSON estructurado según `CuestionarioGenerado` (`evaluacion.py:25-28`, con preguntas `opcion_multiple` de 4 opciones y preguntas `abierta` de análisis de caso).
3. El router invoca `publicar_quiz(preguntas, materia)` (`routers/quiz.py:23-28`), que guarda las preguntas completas (con `respuesta_correcta` y `explicacion`) en `SessionStore.guardar_quiz` (`apps/api/session_store.py:26-27`) y devuelve al navegador solo la copia pública (`pregunta_publica`, `routers/quiz.py:15-20`, sin `respuesta_correcta` ni `explicacion`) junto con `quiz_id`.
4. El estudiante responde cada pregunta en pantalla (`QuizView.tsx:299-362`) y al terminar pulsa «Calificar Quiz» (`handleEvaluar`, líneas 162-187), llamando a `api.evaluarQuiz(quizId, respuestas)` (`client.ts:397-402`) → `POST /api/quiz/evaluar` (`routers/quiz.py:45-57`).
5. El router recupera el quiz de `SessionStore.obtener_quiz(req.quiz_id)`, llama a `EvaluacionService.evaluar_respuestas(preguntas, req.respuestas, materia)` (`core/application/evaluacion.py:158-189`) y elimina el quiz de `SessionStore` (`routers/quiz.py:55`).
6. `EvaluacionService.evaluar_respuestas`:
    - Califica localmente las preguntas `opcion_multiple` comparando la opción elegida con `respuesta_correcta` (`evaluacion.py:165-169`).
    - Envía las preguntas `abierta` a Gemini (`_evaluar_abierta`, `evaluacion.py:72-90`) para obtener `es_correcta` y retroalimentación pedagógica.
    - Registra todos los resultados en el motor de repaso espaciado (`RepasoService.registrar_resultados`, `core/application/repaso.py:43-87`), que crea o actualiza las tarjetas en `repaso/tarjetas.json` con el algoritmo SM-2 y añade un evento `tipo="quiz"` en `repaso/historial.json`.
7. Si el estudiante falló alguna pregunta, `QuizView.tsx:386-399` muestra el botón «Repasar fallos con el tutor», que abre `TutoriaView` pre-configurada con el tema y un enfoque centrado en las preguntas falladas (`App.tsx:214-224`).

- **Flujo alternativo (CLI `ml quiz`):**
    1. El estudiante ejecuta `ml quiz` (`apps/cli/main.py:44-49` → `apps/cli/commands/quiz.py:14-79`).
    2. Elige la asignatura en consola, indica un tema opcional, genera 3 preguntas con `evaluacion_service.generar_quiz`, responde interactivamente en la terminal y recibe la calificación inmediata junto con el resumen de aciertos (`quiz.py:41-74`).
- **Errores:**
    - `cantidad < 1` → `ValueError` en `generar_quiz` (`evaluacion.py:98`) → `HTTP 400`.
    - `quiz_id` inexistente o ya calificado en `SessionStore` → `HTTP 404` (`routers/quiz.py:49`).
- **Datos que lee y escribe:**
    - Lee: notas `.md` en `obsidian_vault`, `plugins/<id>_context.md`, `schedules/<slug>_cronograma.json`, `SessionStore`.
    - Escribe: `repaso/tarjetas.json`, `repaso/historial.json` y estado efímero en `SessionStore`.
- **Resultado esperado:** Cuestionario calificado con retroalimentación por pregunta y tarjetas de repaso espaciado creadas o reprogramadas según aciertos y fallos.
- **Dónde está en el código y en la interfaz:**
    - Interfaz web: `frontend/src/views/QuizView.tsx:41-557`.
    - CLI: `ml quiz` (`backend/src/modular_learning/apps/cli/commands/quiz.py:14-79`).
    - Endpoints: `GET /api/quiz/generar`, `POST /api/quiz/evaluar` (`apps/api/routers/quiz.py:15-57`).
    - Servicio: `EvaluacionService` (`core/application/evaluacion.py:44-189`).

CU-12: Practicar repaso espaciado de tarjetas vencidas, consultar historial y eliminar tarjetas

- **Actor:** Estudiante.
- **Precondiciones:** Tener tarjetas registradas en `repaso/tarjetas.json` (provenientes de quizzes previos, tutorías o guías offline).
- **Flujo principal:**
    1. **Consultar resumen de repaso:** al entrar en «Repaso y Evaluación» (`frontend/src/views/QuizView.tsx:66-92`) o en la portada «Hoy» (`frontend/src/views/HoyView.tsx:104-109, 281-319`), el cliente invoca `api.resumenRepaso()` (`client.ts:404`) → `GET /api/repaso/resumen` (`apps/api/routers/repaso.py:17-21`) → `RepasoService.resumen()` (`core/application/repaso.py:155-163`), que calcula cuántas tarjetas están vencidas (`proximo_repaso <= hoy`), cuántas están `dominadas` (`intervalo_dias >= 21`), cuántas están `en_aprendizaje`, el desglose `por_materia` y los puntos débiles (`temas_debiles` con fallos acumulados, `repaso.py:104-126`).
    2. **Iniciar sesión de repaso de pendientes:** el estudiante pulsa «Repasar pendientes (N)» (`handleIniciarRepaso`, `QuizView.tsx:139-160`), llamando a `api.repasoPendientes(materia, 10)` (`client.ts:406-412`) → `GET /api/repaso/pendientes` (`routers/repaso.py:24-33`) → `RepasoService.preguntas_pendientes` (`repaso.py:128-153`). El servicio convierte hasta `LIMITE_REPASO_DIARIO = 10` tarjetas vencidas en preguntas (si una tarjeta proviene de una tutoría o guía y no tiene `opciones`, la presenta como pregunta `abierta` de síntesis, `repaso.py:139-151`) y el router las publica mediante `publicar_quiz`

(routers/quiz.py:23-28).

3. **Calificar el repaso:** al terminar las tarjetas, QuizView envía las respuestas al mismo endpoint POST /api/quiz/evaluar (CU-11, paso 4), que actualiza los intervalos SM-2 (1 → 3 → round(intervalo * facilidad) días si acierta, o reinicia a 1 día si falla, core/domain/repaso_espaciado.py:74-99).
4. **Consultar historial de sesiones:** en QuizView.tsx:72 o ProgresoView.tsx:46, el cliente invoca api.historialRepaso(materia, limite) (client.ts:414-420) → GET /api/repaso/historial (routers/repaso.py:36-42) → RepasoService.historial (repaso.py:173-176) → JsonHistorialRepo.listar (infrastructure/persistence/json_repaso.py:65-77).
5. **Eliminar una tarjeta de repaso:** el endpoint DELETE /api/repaso/tarjetas/{tarjeta_id} (routers/repaso.py:45-53) → RepasoService.eliminar_tarjeta(tarjeta_id) (repaso.py:178-180) → JsonTarjetasRepo.eliminar (json_repaso.py:42-47) borra una tarjeta específica por su hash id (o responde HTTP 404 si no existe).

- **Flujos alternativos y errores:**
  - Si no hay tarjetas vencidas para hoy, GET /api/repaso/pendientes devuelve quiz_id: "" y preguntas: [] (routers/repaso.py:30-31).
- **Datos que lee y escribe:**
  - Lee y escribe: repaso/tarjetas.json y repaso/historial.json.
- **Resultado esperado:** Tarjetas vencidas repasadas y reprogramadas con sus nuevos intervalos SM-2.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: frontend/src/views/QuizView.tsx:66-92, 139-160, 424-553, frontend/src/views/HoyView.tsx:281-319 y frontend/src/views/ProgresoView.tsx:46, 179-219.
  - Endpoints: GET /api/repaso/resumen, GET /api/repaso/pendientes, GET /api/repaso/historial, DELETE /api/repaso/tarjetas/{tarjeta_id} (apps/api/routers/repaso.py:17-53).
  - Servicio y dominio: RepasoService (core/application/repaso.py:29-180), repaso_espaciado.py (core/domain/repaso_espaciado.py:10-99).

CU-13: Crear una guía de estudio offline y generar sus clases con IA

- **Actor:** Estudiante.
- **Precondiciones:** Tener una asignatura con clases cargadas en su cronograma y clave de Gemini activa.
- **Flujo principal:**
  1. **Listar guías existentes y pendientes:** al abrir «Guías Offline» (frontend/src/views/GuiaView.tsx:251-378), el cliente invoca api.listarGuias() (client.ts:424) → GET /api/guias (apps/api/routers/guias.py:42-44) → GuiaOfflineService.listar() (core/application/guias.py:150-151). Asimismo, en App.tsx:103 y HoyView.tsx:106, se invoca api.guiasPendientes() (client.ts:425) → GET /api/guias/pendientes (routers/guias.py:47-50) → GuiaOfflineService.pendientes_de_confirmar() (guias.py:153-154) para mostrar el contador de guías aún no revisadas.
  2. **Seleccionar clases e iniciar creación:** el estudiante elige una asignatura, marca una o varias clases del cronograma (manualmente o pulsando un botón de corte de parcial en GuiaView.tsx:341-376), escribe un título opcional y opcionalmente indica su presupuesto en minutos (minutosDisponibles), y pulsa «Generar guía» (handleCrearYGenerar, GuiaView.tsx:380-439).
  3. **Crear el esqueleto de la guía:** el cliente llama a api.crearGuia(materia, indices, titulo, minutos) (client.ts:427-439) → POST /api/guias (routers/guias.py:53-59) → GuiaOfflineService.crear (core/application/guias.py:59-117):
     - Verifica que la asignatura tenga cronograma y que indices no esté vacío (guias.py:67-72).
     - Para cada índice válido, construye un objeto ClaseGuia con clase_id, tema, fecha, objetivo, subtemas (uniendo contenido y subtemas_adicionales), estado_previo y contenido_md="" (guias.py:74-96).
     - Calcula el nivel de extension ("completa", "estandar" o "sintesis") llamando a elegir_extension(len(clases_guia), minutos_disponibles) (core/domain/guia_offline.py:194-213).
     - Persiste la guía en guias/guias.json (JsonGuiasRepo.guardar, infrastructure/persistence/json_guias.py:30-33).
  4. **Generar cada clase paso a paso con IA:** a continuación, GuiaView.tsx:403-423 recorre secuencialmente los índices de la guía recién creada e invoca por cada uno api.generarClaseGuia(guia.id, c.indice) (client.ts:442-446) → POST /api/guias/{guia_id}/clases/{indice} (routers/guias.py:91-101) → GuiaOfflineService.generar_clase(guia_id, indice) (core/application/guias.py:119-141):
     - Recupera el contexto del plugin activo de la asignatura (PluginsService.obtener_contexto_plugin).
     - Construye el prompt adaptado a la extension (construir_prompt_guia, guias.py:41-51, que exige incluir <!-- RESPUESTAS --> al final).
     - Genera el texto con LLM_GeminiAdapter.generar_texto y lo guarda en objetivo.contenido_md dentro de guias/guias.json.
  5. También se puede consultar una guía individual mediante api.obtenerGuia(id) (client.ts:440) → GET /api/guias/{guia_id} (routers/guias.py:62-64) → GuiaOfflineService.obtener (guias.py:144-148).
- **Flujos alternativos y errores:**

- Si no se seleccionan clases o la asignatura no tiene cronograma → `ValueError` → `HTTP 400` (`routers/guias.py:58`).
- Si `guia_id` o `indice` no existen al generar una clase → `HTTP 404` (`routers/guias.py:95, 99`).
- **Discrepancia asociada: DISC-10** (los botones de corte de parcial en `GuiaView.tsx:341-376` buscan el `id` de la clase en `p.hasta`, pero `SilabosView.tsx:708-711` guarda una fecha en texto libre).

- **Datos que lee y escribe:**
  - Lee: `schedules/<slug>_cronograma.json`, `plugins/<id>_context.md`.
  - Escribe: `guias/guias.json`. Consume cuota de Gemini por cada clase generada.
- **Resultado esperado:** Guía de estudio creada y persistida en `guias/guias.json` con el desarrollo teórico y las preguntas de autoevaluación de todas las clases seleccionadas.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/GuiaView.tsx:251-677`.
  - Endpoints: `GET /api/guias`, `GET /api/guias/pendientes`, `POST /api/guias`, `GET /api/guias/{guia_id}`, `POST /api/guias/{guia_id}/clases/{indice}` (`apps/api/routers/guias.py:42-64, 91-101`).
  - Servicio y dominio: `GuiaOfflineService` (`core/application/guias.py:52-154`), `guia_offline.py` (`core/domain/guia_offline.py:14-213`).

CU-14: Previsualizar, imprimir y exportar una guía offline en PDF A5

- **Actor:** Estudiante.
- **Precondiciones:** Tener una guía offline en `guias/guias.json` con al menos una clase ya redactada (`contenido_md` no vacío).
- **Flujo principal:**
  1. **Vista previa y modo lectura/impresión:** al abrir una guía en `GuiaView` o pulsar «Vista de lectura / Imprimir» (`frontend/src/views/GuiaImprimirView.tsx:16-143`), el navegador separa cada clase por el marcador `<!-- RESPUESTAS -->` (`separarRespuestas` en `frontend/src/lib/guias.ts:11-22`), muestra el cuerpo de estudio de cada clase, agrupa todas las soluciones al final bajo «Solucionario» y añade una hoja de registro en papel con casillas `Entendida` / `Con dudas` (`GuiaImprimirView.tsx:113-138`), permitiendo invocar `window.print()`.
  2. **Exportación directa a PDF A5 desde el backend:** cuando el estudiante pulsa «Descargar PDF» (`handleDescargarPdf`, `GuiaView.tsx:441-458` o `GuiaImprimirView.tsx:31-46`), el cliente llama a `api.descargarPdfGuia(guia.id)` (`frontend/src/api/client.ts:448-458`) → `GET /api/guias/{guia_id}/pdf` (`backend/src/modular_learning/apps/api/routers/guias.py:75-88`).
  3. El router invoca `GuiaOfflineService.exportar_pdf(guia_id)` (`core/application/guias.py:256-266`) → `ExportadorPdfPyMuPDF.exportar_guia(guia)` (`infrastructure/pdf/exportador_pdf.py:21-110`):
     - Convierte el Markdown de cada clase (`cuerpo_estudio` y `respuestas`) a HTML usando `MarkdownIt("commonmark")` (`exportador_pdf.py:26, 62-86`).
     - Construye un documento HTML completo con portada, índice de clases, desarrollo numerado, solucionario final y tabla de registro en papel (`_construir_html`, `exportador_pdf.py:50-110`).
     - Maqueta el HTML en páginas tamaño **A5** (`420 x 595 pt`, márgenes de `36 pt`) mediante `fitz.Story(html=html, user_css=_CSS)` y `fitz.DocumentWriter` (`exportador_pdf.py:16-17, 28-44`).
  4. El endpoint devuelve los bytes del PDF con `media_type="application/pdf"` y cabecera `Content-Disposition: attachment; filename="guia-<slug>.pdf"` (`routers/guias.py:82-87`), que el navegador descarga automáticamente.
- **Flujos alternativos y errores:**
  - Si `guia_id` no existe → `HTTP 404` (`routers/guias.py:78`).
  - Si la guía aún no tiene ninguna clase generada (`not guia.lista_para_imprimir`) → `ValueError` en `GuiaOfflineService.exportar_pdf` (`guias.py:263-264`) → `HTTP 400` (`routers/guias.py:80`).
- **Datos que lee y escribe:**
  - Lee: `guias/guias.json`. No modifica ningún dato persistido.
- **Resultado esperado:** Descarga de un archivo `.pdf` en formato cuaderno A5 con las clases y el solucionario separado al final.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/GuiaView.tsx:441-458`, `frontend/src/views/GuiaImprimirView.tsx:16-143`, `frontend/src/lib/guias.ts:11-72`.
  - Endpoint: `GET /api/guias/{guia_id}/pdf` (`apps/api/routers/guias.py:75-88`).
  - Servicio y adaptador: `GuiaOfflineService.exportar_pdf` (`core/application/guias.py:256-266`), `ExportadorPdfPyMuPDF` (`infrastructure/pdf/exportador_pdf.py:20-110`).

CU-15: Confirmar la revisión de una guía offline al volver

- **Actor:** Estudiante.
- **Precondiciones:** Tener una guía offline generada y haber estudiado sus clases fuera de línea.

- **Flujo principal:**
  1. En la vista de revisión de una guía (`frontend/src/views/RevisionGuiaView.tsx:18-198`, accesible desde «Guías Offline» o desde el aviso «Al volver: confirmar guías pendientes» en `HoyView.tsx:261-279`), el estudiante marca para cada clase uno de los tres estados (`"entendida"`, `"con_dudas"` o `"pendiente"`) y opcionalmente escribe sus dudas o apuntes en el campo de texto (`RevisionGuiaView.tsx:120-175`). Mientras edita, los cambios se guardan automáticamente como borrador en `localStorage` (`guardarBorradorRevision` en `frontend/src/lib/guias.ts:87-96`).
  2. Al pulsar «Confirmar revisión» (`handleConfirmar`, `RevisionGuiaView.tsx:53-77`), el cliente invoca `api.revisarGuia(guia.id, revisiones)` (`frontend/src/api/client.ts:460-467`) → `POST /api/guias/{guia_id}/revision` (`backend/src/modular_learning/apps/api/routers/guias.py:104-111`) → `GuiaOfflineService.registrar_revision(guia_id, revisiones)` (`core/application/guias.py:157-212`).
  3. Por cada clase incluida en `revisiones`:
     - Actualiza `clase.resultado` (`"entendida"`, `"con_dudas"`, `"pendiente"`) y `clase.notas_revision` (`guias.py:181-182`).
     - Si `resultado == "entendida"`:
       - Marca la clase como `"completado"` en el cronograma por `clase_id` (o por `tema` si no tiene `clase_id`) con `JsonCronogramaRepo.marcar_clase_completada` (`guias.py:184-187`).
       - Registra la clase en el repaso espaciado con `RepasoService.registrar_tutoria(guia.materia, clase.tema, clase.subtemas)` (`guias.py:188-193`), creando tarjetas dominadas (`intervalo_dias = 21`) y un evento en `repaso/historial.json`.
     - Si `resultado != "entendida"` pero anteriormente estaba `"entendida"`, restaura `clase.estado_previo` en el cronograma (`guias.py:194-200`).
  4. Marca `guia.revisada = True`, asigna `guia.fecha_revision = date.today().isoformat()`, guarda la guía en `guias/guias.json` (`guias.py:202-204`) y añade un evento global `tipo="guia"` en `repaso/historial.json` (`guias.py:205-211`).
  5. En el navegador se limpia el borrador de `localStorage` (`limpiarBorradorRevision`, `RevisionGuiaView.tsx:66`). Si alguna clase quedó marcada `"con_dudas"`, la vista ofrece un botón directo «Abrir en tutoría» que lanza `TutoriaView` con esa asignatura, `clase_id` y las dudas anotadas como enfoque (`RevisionGuiaView.tsx:161-172`).
- **Flujos alternativos y errores:**
  - Si `guia_id` no existe → `HTTP 404` (`routers/guias.py:110`).
  - Si se envía un `resultado` inválido → `ValueError` en `GuiaOfflineService.registrar_revision` (`guias.py:173-174`) → `HTTP 400` (`routers/guias.py:108`).
- **Datos que lee y escribe:**
  - Lee y escribe: `guias/guias.json`, `schedules/<slug>_cronograma.json`, `repaso/tarjetas.json`, `repaso/historial.json` y `localStorage`.
- **Resultado esperado:** Guía marcada como revisada, clases entendidas completadas en el cronograma y registradas en el repaso espaciado.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/RevisionGuiaView.tsx:18-198`, `frontend/src/lib/guias.ts:74-108`.
  - Endpoint: `POST /api/guias/{guia_id}/revision` (`apps/api/routers/guias.py:104-111`).
  - Servicio: `GuiaOfflineService.registrar_revision` (`core/application/guias.py:157-212`).

CU-16: Deshacer la confirmación de una guía offline o eliminar la guía

- **Actor:** Estudiante.
- **Precondiciones:** Tener una guía offline creada (y en el caso de deshacer, previamente confirmada).
- **Flujo principal:**
  1. **Deshacer confirmación:** en `RevisionGuiaView.tsx:79-95` (o `GuiaView`), el estudiante pulsa «Deshacer confirmación», lo que invoca `api.deshacerRevisionGuia(guia.id)` (`frontend/src/api/client.ts:469-473`) → `POST /api/guias/{guia_id}/deshacer` (`backend/src/modular_learning/apps/api/routers/guias.py:114-120`) → `GuiaOfflineService.deshacer_revision(guia_id)` (`core/application/guias.py:214-250`):
     - Por cada `ClaseGuia` cuyo `resultado` era `"entendida"`, restaura en el cronograma el estado que tenía antes de revisar la guía (`clase.estado_previo`, `"pendiente"` o `"completado"`) llamando a `JsonCronogramaRepo.marcar_clase` por `clase_id` o `tema` (`guias.py:222-238`).
     - Reinicia `clase.resultado = "pendiente"` y `clase.notas_revision = ""` en todas las clases de la guía (`guias.py:239-240`).
     - Pone `guia.revisada = False` y `guia.fecha_revision = None`, y guarda en `guias/guias.json` (`guias.py:241-243`).
  2. **Eliminar una guía:** el estudiante pulsa el botón de eliminar guía (`GuiaView.tsx:460-476`), llamando a `api.eliminarGuia(id)` (`client.ts:441`) → `DELETE /api/guias/{guia_id}` (`routers/guias.py:67-72`) → `GuiaOfflineService.eliminar_guia(guia_id)` (`guias.py:252-254`) → `JsonGuiasRepo.eliminar(guia_id)` (`infrastructure/persistence/json_guias.py:35-40`), que borra la guía de `guias/guias.json`.

- **Flujos alternativos y errores:**
  - Si `guia_id` no existe en cualquiera de las dos operaciones → `HTTP 404` (`routers/guias.py:70, 118`).
- **Datos que lee y escribe:**
  - Lee y escribe: `guias/guias.json` y `schedules/<slug>_cronograma.json`.
- **Resultado esperado:** Al deshacer, la guía vuelve al estado pendiente de confirmar y las clases del cronograma recuperan su `estado_previo`; al eliminar, la guía desaparece de `guias/guias.json`.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/RevisionGuiaView.tsx:79-95`, `frontend/src/views/GuiaView.tsx:460-476`.
  - Endpoints: `POST /api/guias/{guia_id}/deshacer`, `DELETE /api/guias/{guia_id}` (`apps/api/routers/guias.py:67-72, 114-120`).
  - Servicio: `GuiaOfflineService.deshacer_revision` y `eliminar_guia` (`core/application/guias.py:214-254`).

CU-17: Configurar y actualizar plugins prácticos de contexto técnico (Web, CLI `ml up` y `snapshot.py`)

- **Actor:** Estudiante (en modo local o híbrido para escanear rutas del disco).
- **Precondiciones:** Contar con archivos de laboratorio o proyectos (`.pkt`/`.pka` para Cisco, `.json` para Node-RED o carpeta de código para Web).
- **Flujo principal (Interfaz Web):**
  1. **Listar plugins y su estado:** al abrir la vista «Plugins» (`frontend/src/views/PluginsView.tsx:45-64`), el cliente invoca `api.obtenerPlugins()` (`client.ts:475`) → `GET /api/plugins` (`apps/api/routers/plugins.py:23-30`) → `PluginsService.listar_info_plugins()` (`core/application/plugins.py:95-123`), que devuelve para cada plugin registrado (`cisco`, `nodered`, `web`) su `id`, `nombre`, `materia_defecto`, `extensiones`, `materia_configurada` (leída de `config/plugins_config.json`) y `tiene_contexto` (si `plugins/<id>_context.md` tiene contenido).
  2. **Vincular un plugin a una asignatura:** el estudiante selecciona una asignatura en el desplegable de la tarjeta del plugin (`handleCambiarMateria`, `PluginsView.tsx:66-85`), llamando a `api.configurarPlugin(pluginId, materia)` (`client.ts:477-482`) → `POST /api/plugins/config` (`routers/plugins.py:33-39`) → `PluginsService.guardar_configuracion_plugin(plugin_id, materia)` (`plugins.py:86-93`) → `JsonPluginConfigRepo.guardar_configuracion` (`json_repositorios.py:398-412`), que persiste el vínculo en `config/plugins_config.json`.
  3. **Analizar carpeta o archivo y actualizar la memoria del plugin:** el estudiante introduce la ruta local del laboratorio o proyecto y pulsa «Sincronizar Contexto» (`handleActualizar`, `PluginsView.tsx:87-115`), invocando `api.actualizarPlugin(pluginId, ruta)` (`client.ts:484-489`) → `POST /api/plugins/actualizar` (`routers/plugins.py:42-60`) → `PluginsService.actualizar_plugin(plugin_id, ruta)` (`core/application/plugins.py:177-281`):
     - Si `plugin_id == "cisco"`, llama a `CiscoPlugin.analizar_carpeta` (`infrastructure/plugins/cisco.py:261-302`) o `analizar_topologia` + `LectorDocumentosLocal.extraer_texto`, sintetiza el escenario con Gemini (`plugins.py:209-219`) y guarda el resultado en `plugins/cisco_context.md`.
     - Si `plugin_id == "nodered"`, llama a `NodeRedPlugin.analizar_carpeta` o `analizar_flujo` (`infrastructure/plugins/nodered.py:63-214`), sintetiza la arquitectura IoT con Gemini (`plugins.py:242-252`) y guarda en `plugins/nodered_context.md`.
     - Si `plugin_id == "web"`, llama a `WebPlugin.analizar_proyecto` (`infrastructure/plugins/web.py:336-394`), sintetiza la arquitectura de software con Gemini (`plugins.py:265-275`) y guarda en `plugins/web_context.md`.
- **Flujo alternativo 1 (CLI `ml up`):**
  1. El estudiante ejecuta `ml up` (`apps/cli/main.py:60-65` → `apps/cli/commands/up.py:15-54`), elige uno de los plugins registrados en `plugins_service.obtener_plugins()`, introduce por consola la ruta del archivo o carpeta y se ejecuta `plugins_service.actualizar_plugin(plugin_id, ruta_limpia)`.
- **Flujo alternativo 2 (Script CLI `snapshot.py`):**
  1. El módulo `backend/src/modular_learning/apps/cli/commands/snapshot.py:10-125` permite recorrer una carpeta de código local (`.py`, `.js`, `.ts`, `.html`, `.css`, `.sql`, `.json`, `.md`), concatenar el árbol de directorios y el contenido de los archivos (truncando a 15.000 caracteres) y guardarlo directamente con `plugins_service.guardar_contexto_extraido(plugin.id, reporte)`. *(Véase **DISC-12**: este módulo no está conectado como comando en `apps/cli/main.py`).*
- **Errores:**
  - `plugin_id` no válido o `ruta` inexistente en el sistema de archivos del servidor → `HTTP 400` / `HTTP 404` (`routers/plugins.py:45-58`).
- **Datos que lee y escribe:**
  - Lee: archivos `.pkt`, `.pka`, `.json` o código fuente en la ruta local indicada; `config/plugins_config.json`.
  - Escribe: `config/plugins_config.json` y `plugins/<id>_context.md`. Consume cuota de Gemini.
- **Resultado esperado:** Memoria técnica del plugin actualizada en `plugins/<id>_context.md` y lista para inyectarse automáticamente en las sesiones de tutoría, quizzes y guías offline de la asignatura vinculada.
- **Dónde está en el código y en la interfaz:**

- Interfaz web: `frontend/src/views/PluginsView.tsx:16-247`.
- CLI: `ml up` (`apps/cli/commands/up.py:15-54`) y `apps/cli/commands/snapshot.py:10-125`.
- Endpoints: `GET /api/plugins`, `POST /api/plugins/config`, `POST /api/plugins/actualizar` (`apps/api/routers/plugins.py:23-60`).
- Servicio y adaptadores: `PluginsService` (`core/application/plugins.py:30-281`), `CiscoPlugin` (`infrastructure/plugins/cisco.py`), `NodeRedPlugin` (`infrastructure/plugins/nodered.py`), `WebPlugin` (`infrastructure/plugins/web.py`).

CU-18: Consultar y guardar ajustes del sistema, apariencia y vaciado de datos

- **Actor:** Estudiante.
- **Precondiciones:** Servidor `ml-web` en ejecución.
- **Flujo principal:**
  1. **Consultar ajustes actuales:** al abrir la vista «Ajustes» (`frontend/src/views/ConfigView.tsx:66-85`) o al iniciar otras vistas que consultan `en_nube`, el cliente llama a `api.obtenerConfig()` (`frontend/src/api/client.ts:144`) → `GET /api/config` (`backend/src/modular_learning/apps/api/routers/config.py:17-23`) → `ConfiguracionService.obtener_ajustes()` (`core/application/configuracion.py:25-26`) → `JsonAjustesRepo.obtener_ajustes()` (`infrastructure/persistence/json_ajustes.py:39-46`). El router añade el booleano `en_nube = os.getenv("ML_EN_NUBE") == "1"` (`routers/config.py:21`).
  2. **Guardar rutas y clave de Gemini (solo en modo local/híbrido):** cuando `en_nube` es `false`, el estudiante puede editar `obsidian_vault`, `silabos_dir`, `material_dir` y `gemini_api_key` y pulsar «Guardar Ajustes» (`handleGuardar`, `ConfigView.tsx:87-115`). Se invoca `api.guardarConfig(datos)` (`client.ts:145-150`) → `POST /api/config` (`routers/config.py:26-36`) → `ConfiguracionService.guardar_ajustes` (`configuracion.py:28-31`) → `JsonAjustesRepo.guardar_ajustes` (`json_ajustes.py:48-63`), que escribe `config/ajustes.json` y actualiza en caliente las propiedades de `settings` en memoria (`settings.py:26-73`). Si `en_nube` es `true`, `ConfigView.tsx:214-220` oculta estos campos e informa que se administran por variables de entorno en el servidor.
  3. **Cambiar apariencia (tema claro / oscuro):** en `ConfigView.tsx:188-211` o desde el botón del pie de la barra lateral (`App.tsx:379-388`), el estudiante alterna entre modo oscuro y claro, persistiendo `ml_theme` en `localStorage` y aplicando la clase `.light` en `document.documentElement` (`App.tsx:86-98`).
  4. **Zona de peligro — Vaciar datos del semestre desde Ajustes:** en la parte inferior de `ConfigView.tsx:336-445`, el estudiante dispone de tres acciones de limpieza con confirmación:
     - **Vaciar cronogramas:** recorre `api.obtenerMaterias()` y llama a `api.eliminarCronograma(m)` por cada asignatura (`ConfigView.tsx:117-146`). *(Afectado por **DISC-02** / **P13**: actualmente esto borra también las asignaturas).*
     - **Vaciar materiales ingestados:** llama a `api.vaciarMateriales()` (`POST /api/material/vaciar`, `ConfigView.tsx:148-157`).
     - **Reiniciar semestre completo:** ejecuta `api.vaciarMateriales()` y luego elimina todas las asignaturas con `api.eliminarMateria(m.slug)` (`ConfigView.tsx:159-177`).
- **Flujos alternativos y errores:**
  - Si falla la escritura en `AlmacenDocumentos`, el cliente captura el error y muestra una alerta roja (`ConfigView.tsx:108-111`).
- **Datos que lee y escribe:**
  - Lee y escribe: `config/ajustes.json` y `localStorage` (`ml_theme`). En las acciones de vaciado modifica `config/materias.json`, `schedules/*_cronograma.json`, `config/registro_materiales.json`, `config/proyectos.json` y las notas en `obsidian_vault`.
- **Resultado esperado:** Rutas de trabajo, clave de API y tema visual actualizados inmediatamente sin necesidad de reiniciar el servidor.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/ConfigView.tsx:26-449`, `frontend/src/App.tsx:86-98`.
  - Endpoints: `GET /api/config`, `POST /api/config` (`apps/api/routers/config.py:17-36`).
  - Servicio y repositorio: `ConfiguracionService` (`core/application/configuracion.py:17-31`), `JsonAjustesRepo` (`infrastructure/persistence/json_ajustes.py:25-63`).

CU-19: Consultar estadísticas de progreso, portada «Hoy» y comprobación de salud (`HoyView`, `ProgresoView`, `ml stats`, `/api/salud`)

- **Actor:** Estudiante o servicio de monitoreo (Render healthcheck).
- **Precondiciones:** Ninguna (para `/api/salud` ni siquiera se requiere autenticación HTTP Basic).
- **Flujo principal:**
  1. **Portada diaria («Hoy», `frontend/src/views/HoyView.tsx:88-365`):** al abrir la aplicación, `HoyView` agrega en una sola pantalla el estado del día consultando en paralelo (`HoyView.tsx:104-115`):
     - `GET /api/materias/cronogramas` (`api.obtenerCronogramas`): calcula cuál es la **próxima clase pendiente** priorizando la asignatura con menor porcentaje de avance (`HoyView.tsx:194-213`) y ofrece los botones «Estudiar con tutor» y «Preparar guía offline» (`líneas 247-259`).
     - `GET /api/guias/pendientes` (`api.guiasPendientes`): muestra el bloque «Al volver: confirmar guías pendientes» con acceso directo a la revisión de cada guía (`líneas 261-279`).

- `GET /api/repaso/resumen` (`api.resumenRepaso`): muestra cuántas tarjetas de repaso vencen hoy y los puntos débiles detectados (`líneas 281-319`).
- `GET /api/material/escanear` (`api.escanearMaterial`): muestra en la columna derecha si hay archivos nuevos pendientes de ingestar (`líneas 321-356`).

2. **Dashboard de estadísticas (`frontend/src/views/ProgresoView.tsx:31-235` y CLI `ml stats`):**
   - En la web, `ProgresoView` llama a `api.obtenerStats()` (`client.ts:491`) → `GET /api/stats` (`apps/api/routers/stats.py:15-19`), además de `api.resumenRepaso()` y `api.historialRepaso(undefined, 20)`.
   - En la CLI, `ml stats` (`apps/cli/main.py:52-57` → `apps/cli/commands/stats.py:15-68`) invoca directamente `TelemetriaService.obtener_estadisticas()`.
   - `TelemetriaService.obtener_estadisticas()` (`core/application/telemetria.py:36-69`) recorre las asignaturas de `JsonMateriaRepo.cargar_materias()`, cuenta en cada cronograma el total de clases y las clases con `estado == "completado"`, calcula el porcentaje por asignatura y el `porcentaje_global`, y verifica qué plugins tienen memoria activa (`PluginsService.obtener_contexto_plugin`).
3. **Comprobación de salud del servidor (`GET /api/salud`):**
   - Definido en `backend/src/modular_learning/apps/api/main.py:57-60`. Devuelve `{"estado": "ok"}` con código `HTTP 200` y está exento del middleware de control de acceso (`apps/api/acceso.py:39-40`) para que plataformas PaaS como Render verifiquen que el contenedor está vivo.

- **Flujos alternativos y errores:**
  - Si no existen asignaturas ni cronogramas, `obtener_estadisticas` devuelve contadores en `0` (`telemetria.py:39-45`).
  - **Discrepancias asociadas: DISC-08** (`HoyView` escanea carpetas locales aunque `en_nube` sea `true`) y **DISC-12** (`README.md` y `docs/ARCHITECTURE.md` mencionan cálculo de «rachas de estudio» en `ml stats`/`TelemetriaService`, pero el servicio no calcula rachas).
- **Datos que lee y escribe:**
  - Lee: `config/materias.json`, `schedules/*_cronograma.json`, `plugins/*_context.md`, `repaso/tarjetas.json`, `repaso/historial.json`, `guias/guias.json`. No escribe datos.
- **Resultado esperado:** Visión consolidada del avance curricular del semestre, métricas de repaso espaciado y estado de salud del servidor.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/HoyView.tsx:88-365`, `frontend/src/views/ProgresoView.tsx:31-235`.
  - CLI: `ml stats` (`backend/src/modular_learning/apps/cli/commands/stats.py:15-68`).
  - Endpoints: `GET /api/stats` (`apps/api/routers/stats.py:15-19`), `GET /api/salud` (`apps/api/main.py:57-60`).
  - Servicio: `TelemetriaService` (`core/application/telemetria.py:18-69`).

CU-20: Migrar datos entre versiones y sincronizar datos locales hacia Postgres en la nube (`scripts/migrar_datos.py` y `deploy/subir_datos.py`)

- **Actor:** Estudiante / Administrador del despliegue.
- **Precondiciones:** Acceso por terminal al entorno Python del proyecto; para `subir_datos.py`, tener definida la variable `DATABASE_URL` apuntando a PostgreSQL (Neon).
- **Flujo principal:**
  1. **Migración y saneamiento idempotente de datos (`scripts/migrar_datos.py:23-240`):**
     - Se ejecuta con `python scripts/migrar_datos.py` (o `--simular` para no escribir cambios). Opera sobre `crear_almacen()` (`bootstrap.py:44-56`), por lo que funciona tanto contra `data/` local como contra Postgres si está definido `DATABASE_URL`.
     - **Paso 1 (`migrar_materias`, líneas 28-69):** lee `config/materias.json` (convirtiendo listas antiguas de strings a objetos `{slug, nombre}` deduplicados por `slug_materia`) e incorpora cualquier asignatura que tenga archivo en `schedules/*_cronograma.json` pero falte en el catálogo.
     - **Paso 2 (`migrar_cronogramas`, líneas 72-145):** renombra las claves de cronogramas antiguos para que usen `<slug>_cronograma.json`, fusiona cronogramas duplicados (conservando el que tenga más clases completadas o más clases en total), corrige el campo `materia` al nombre canónico del catálogo y asegura que cada clase tenga un `id` único con `asegurar_ids_clases`.
     - **Paso 3 (`migrar_registro_materiales`, líneas 148-174):** normaliza el campo `materia` de todas las entradas de `config/registro_materiales.json` al nombre del catálogo.
     - **Paso 4 (`migrar_plugins_config`, líneas 177-202):** normaliza el campo `materia` de `config/plugins_config.json`.
  2. **Subida de datos locales a Neon (`deploy/subir_datos.py:45-76`):**
     - Se ejecuta con `DATABASE_URL="postgresql://..." python deploy/subir_datos.py` (en modo vista previa) o añadiendo `--subir` para escribir en la base de datos.
     - Recorre todas las claves de `AlmacenArchivos(data_dir)` (`subir_datos.py:57`).
     - Al subir `config/ajustes.json`, reemplaza las rutas del PC por las rutas internas del contenedor Docker en Render

(`AJUSTES_NUBE` en `subir_datos.py:21-30`: `obsidian_vault="/tmp/ml-boveda"`, `silabos_dir="/app/data/silabos"`, `material_dir="/app/data/material"`, `gemini_api_key=""`).
    - Escribe cada documento en `AlmacenPostgres(url)` mediante `remoto.escribir_texto(clave, contenido)` (`subir_datos.py:68-71`).
- **Flujos alternativos y errores:**
    - Si no se define `DATABASE_URL` al ejecutar `deploy/subir_datos.py`, termina con código de error `2` indicando el comando exacto (`subir_datos.py:49-52`).
- **Datos que lee y escribe:**
    - `migrar_datos.py`: lee y escribe `config/materias.json`, `schedules/*_cronograma.json`, `config/registro_materiales.json`, `config/plugins_config.json`.
    - `subir_datos.py`: lee todos los archivos de `data/` y escribe todas las filas en la tabla `documentos` de Postgres.
- **Resultado esperado:** Datos locales y remotos normalizados bajo el esquema `slug`/`id` actual y sincronizados en PostgreSQL para el despliegue en Render o uso híbrido.
- **Dónde está en el código y en la interfaz:**
    - Scripts CLI: `scripts/migrar_datos.py:1-240` y `deploy/subir_datos.py:1-76`.

---

5. Flujos de extremo a extremo del semestre

5.1. Flujo A: Empezar el semestre (Configuración inicial y planificación)

1. **Configurar rutas y API Key (`CU-18`):** el estudiante abre «Ajustes» (`ConfigView`), define las rutas locales de su bóveda de Obsidian (`obsidian_vault`), su carpeta de sílabos (`silabos_dir`) y su carpeta de materiales (`material_dir`), e introduce su `gemini_api_key`.
2. **Dar de alta las asignaturas (`CU-01`):** en la vista «Asignaturas» (`AsignaturasView`), crea las materias del semestre verificando que sus nombres tengan la ortografía y tildes definitivas.
3. **Importar los sílabos oficiales (`CU-02`):** desde «Sílabos y Cronogramas» (`SilabosView`) o ejecutando `ml es` en consola, extrae con IA las sesiones de cada sílabo `.pdf`/`.docx`, revisa la previsualización y guarda el cronograma de cada asignatura.
4. **Ajustar clases y cortes de parcial (`CU-03`):** en «Sílabos y Cronogramas», edita o añade sesiones si el profesor hizo cambios al programa, define los cortes de parcial (`Parcial 1`, `Parcial 2`) y comprueba la línea de tiempo global en la pestaña «Calendario».
5. **Vincular plugins prácticos (`CU-17`):** en la vista «Plugins» (`PluginsView`), asocia cada plugin (`cisco`, `nodered`, `web`) con su asignatura correspondiente para que el tutor disponga de contexto práctico durante el curso.
6. **Publicar datos en la nube si usa acceso móvil (`CU-20`):** si además despliega la aplicación en Render + Neon, ejecuta `python deploy/subir_datos.py --subir` para copiar el catálogo y los cronogramas iniciales a PostgreSQL.

5.2. Flujo B: Una semana normal de clases (Ingesta, estudio y repaso continuo)

1. **Revisar qué toca hoy (`CU-19`):** al abrir la aplicación en la vista «Hoy» (`HoyView`), el estudiante identifica la próxima clase pendiente sugerida, las guías offline pendientes de confirmar y las tarjetas de repaso vencidas.
2. **Ingestar los archivos nuevos de la semana (`CU-05` / `CU-06`):**
    - Si está en su PC (modo local o híbrido), coloca los documentos de la semana en `<material_dir>/<Asignatura>/` y pulsa «Revisar carpetas» → «Procesar pendientes» (`CU-05`, o ejecuta `ml im`), generando los apuntes en Obsidian y alineándolos con la clase correspondiente del cronograma.
    - Si está desde otro dispositivo o tiene un documento puntual, lo sube desde «Subir un archivo suelto» (`CU-06`).
3. **Estudiar la clase de la semana (dos modalidades complementarias):**
    - **Modalidad interactiva en pantalla (`CU-10`):** pulsa «Estudiar con tutor» desde «Hoy» (`TutoriaView` o `ml init`), recorre la explicación en *streaming* de cada subtema, plantea dudas en el chat lateral y al terminar pulsa «Finalizar y Guardar en Obsidian», lo que genera el resumen en su bóveda, marca la clase como completada y programa tarjetas de repaso.
    - **Modalidad desconectada / en papel (`CU-13` → `CU-14` → `CU-15`):** selecciona las clases de la semana en «Guías Offline» (`GuiaView`), indica cuántos minutos tiene disponibles, genera la guía con IA (`CU-13`), descarga el cuaderno en **PDF A5** (`CU-14`) para estudiar sin pantallas y, al regresar, abre la revisión (`RevisionGuiaView`, `CU-15`) para marcar qué clases quedaron `Entendidas` y cuáles `Con dudas` (abriendo estas últimas directamente en la tutoría).
4. **Consolidar la memoria a largo plazo (`CU-11` y `CU-12`):** cierra la semana en «Repaso y Evaluación» (`QuizView`) repasando las tarjetas vencidas del día (`CU-12`) o generando un quiz nuevo sobre el tema recién estudiado (`CU-11`).

5.3. Flujo C: Preparación antes de un examen parcial

1. **Sincronizar entregables y laboratorios prácticos (`CU-08` y `CU-17`):**
    - En «Material de Estudio» (`MaterialView`), escanea e ingesta o actualiza las fichas técnicas de los proyectos del parcial (`CU-08`).
    - En «Plugins» (`PluginsView` o `ml up`), actualiza el contexto de las topologías de Packet Tracer (`cisco`), flujos de Node-RED (`nodered`) o código fuente (`web`) para que las preguntas y explicaciones reflejen el estado real del proyecto (`CU-17`).
2. **Generar un cuaderno de repaso de todo el parcial (`CU-13` y `CU-14`):** en «Guías Offline» (`GuiaView`), selecciona todas las clases

pendientes del corte del parcial, genera una guía de extensión adaptada y exporta el PDF A5 con solucionario al final para repasar todo el bloque temático.

3. **Detectar puntos débiles y entrenar con quizzes (CU-12, CU-11 y CU-19):**
   - En «Hoy» (`HoyView`) o «Progreso» (`ProgresoView`), consulta los `temas_debiles` (aquellos con más fallos acumulados en `RepasoService`).
   - Ejecuta sesiones de repaso espaciado (`CU-12`) y genera quizzes específicos (`CU-11`) para cada tema del parcial.
4. **Reforzar los fallos con el tutor (CU-10):** tras calificar cada quiz, pulsa el botón «Repasar fallos con el tutor» (`QuizView.tsx:386-399`), que inicia una sesión de tutoría enfocada exclusivamente en los conceptos errados antes del examen.

---

6. Discrepancias detectadas y matriz de trazabilidad

6.1. Tabla de discrepancias (Comportamiento esperado vs. Comportamiento actual del código)

Cada discrepancia ha sido verificada directamente en el código fuente citando `archivo:línea` y contrastada con la sección «Comportamiento esperado» de `docs/PENDIENTES.md` (tarea D1), `README.md`, `docs/ARCHITECTURE.md` y `docs/DESPLIEGUE.md`.

ID	Caso de uso	Comportamiento esperado	Comportamiento actual según el código (`archivo:línea`)	Gravedad	Tarea en `PENDIENTES.md`
DISC-01	`CU-01`, `CU-02`	Al crear o importar una asignatura con tildes en `SilabosView`, la lista del catálogo debe normalizar las claves sin perder tildes y dejar seleccionada la asignatura recién creada o importada.	En `frontend/src/views/SilabosView.tsx:88` y `472`, al guardar un sílabo ya se cruza con `buscarMateriaPorClave(lista, seleccionar)`. Sin embargo: 1) en `handleCrearMateria` (`SilabosView.tsx:211`) se invoca `await cargarCatalogo()` **sin** pasar `nombreNuevaMateria.trim()`, por lo que crear una asignatura desde el modal de `SilabosView` no la selecciona si ya había otra activa (`línea 94`); y 2) el *fallback* cuando `res.catalogo` no viene poblado (`SilabosView.tsx:82` y `135`) calcula el slug con `m.toLowerCase().replace(/\s+/g, '-')` sin quitar diacríticos (`normalize('NFD')`).	Media	**P12** (parcialmente relacionada)
DISC-02	`CU-04`, `CU-18`	Vaciar el cronograma debe dejar la asignatura en el catálogo sin clases, tal como promete el diálogo de confirmación en `SilabosView.tsx:248` (*«Se borrarán todas sus clases pero se conservará la asignatura en el catálogo»*).	`DELETE /api/materias/{materia}/cronograma` (`backend/src/modular_learning/apps/api/routers/materias.py:86`) invoca `servicio.eliminar_cronograma(materia, borrar_materia=True)`. En `core/application/materias.py:87-94`, `borrar_materia=True` ejecuta `self.eliminar_materia(materia, borrar_cronograma=False)`, lo que **elimina la asignatura de `config/materias.json` y borra además sus materiales de `config/registro_materiales.json`** (`materias.py:65-66`). Adicionalmente, `SilabosView.tsx:255-257` solo hace `setClasesCronograma([])` sin recargar el catálogo, dejando seleccionada en pantalla una asignatura ya inexistente en el backend.	**Alta**	**P13**
DISC-03	`CU-01`, `CU-02`, `CU-06`, `CU-10`	Una asignatura escrita con otra grafía (mayúsculas, sin tildes) es la misma asignatura; el nombre visible del catálogo no debe cambiar salvo al renombrar explícitamente, y las notas en Obsidian deben ir siempre a la carpeta con el nombre canónico del catálogo.	1) `JsonMateriaRepo.agregar_materia` (`infrastructure/persistence/json_repositorios.py:87-89`) llama a `self.guardar(Materia.desde_nombre(nueva_materia.strip()))`, y `guardar()` (`json_repositorios.py:67-69`) **sobrescribe** `materias[i] = materia` cuando coincide el `slug`. Por ello, guardar un sílabo (`core/application/silabo.py:124`), subir un material (`apps/api/routers/material.py:106`) o iniciar una tutoría (`apps/api/routers/tutoria.py:55`) con `"INGENIERIA DE SOFTWARE II"` pisa el nombre `"Ingeniería de Software II"` del catálogo. 2) `SilaboService.guardar_cronograma` (`silabo.py:119-125`), `IngestaService.ingestar_archivo` (`core/application/ingesta.py:104-134`) y `TutoriaService.iniciar_sesion` (`core/application/tutoria.py:115-165`) no resuelven primero el nombre contra `JsonMateriaRepo.obtener_por_slug`. En consecuencia, `ingestar_archivo` (`ingesta.py:132-134`) y `finalizar_sesion` (`tutoria.py:269`) escriben notas de Obsidian bajo `<obsidian_vault>/<texto_enviado>/`, creando carpetas duplicadas si varía la escritura.	**Alta**	**P14**
DISC-04	`CU-02`, `CU-06`	Al subir archivos en la web (`SilabosView` y `MaterialView`), la asignatura debe elegirse de una lista desplegable del catálogo en lugar de escribirse a mano.	En `frontend/src/views/MaterialView.tsx:758-764` (subir archivo suelto) y en `frontend/src/views/SilabosView.tsx:972-977` (subir sílabo), el campo «Asignatura» es un `<Input>` de texto libre. Además, al seleccionar un archivo en `SilabosView.tsx:420-427`, se autocompleta `subidaMateria` con el nombre crudo derivado del archivo sin cruzarlo con el catálogo mediante `clave()`.	Media	**P15**

DISC-05	CU-05, CU-06	Las presentaciones `.pptx` deben poder ingestarse (incluidas las que contienen diapositivas con imágenes), tal como exige la lista de comportamiento esperado de D1 y como afirman `README.md:5, 13` y `docs/ARCHITECTURE.md:44, 84`.	En `backend/src/modular_learning/core/application/ingesta.py:23-24`, `EXTENSIONES_MATERIAL = (".pdf", ".docx", ".txt", ".md")` y `EXTENSIONES_NO_SOPORTADAS_WEB = (".pptx",)`. Durante el escaneo (`ingesta.py:224-226`), cualquier `.pptx` se clasifica como `no_soportados` y se ignora (tanto en web como en CLI `ml im`). En `POST /api/material/ingestar` (`apps/api/routers/material.py:97-101`), subir un `.pptx` devuelve `HTTP 400`. Tampoco existe lectura ni conversión de `.pptx` a PDF en `LectorDocumentosLocal.extraer_texto` (`infrastructure/documents/lector_documentos.py:40-61`).	**Alta**	**P16**
DISC-06	CU-05, CU-06, CU-07	En la pestaña «Material de Estudio», lo principal es revisar el historial de lo ingestado; subir un archivo suelto es ocasional (colapsado por defecto) y los resultados del escaneo solo deben mostrarse si se ha escaneado en la visita actual.	En `frontend/src/views/MaterialView.tsx:704-921`, el orden actual en la columna derecha es: 1) Resultados del escaneo (`líneas 707-741`), 2) Tarjeta «Subir un archivo suelto» siempre abierta (`líneas 744-842`), y 3) Tabla «Materiales Ingestados» relegada al final de la página (`líneas 845-920`).	Media	**P17**
DISC-07	CU-05, CU-06	`README.md:5, 13` y `docs/ARCHITECTURE.md:84, 93` documentan «extracción nativa de diagramas / imágenes embebidas desde `.docx`» mediante el lector multimodal.	`LectorDocumentosLocal.extraer_texto` (`backend/src/modular_learning/infrastructure/documents/lector_documentos.py:51-55`) solo ejecuta `"\n".join(p.text for p in documento.paragraphs)` con `python-docx`. Ignora por completo tanto las imágenes/diagramas embebidos como el texto dentro de tablas (`documento.tables`), y `IngestaService._generar_contenido` (`core/application/ingesta.py:77-79`) envía los `.docx` a `generar_texto` (solo texto plano), reservando `procesar_documento_multimodal` exclusivamente para `.pdf`.	Media	No registrada (candidata nueva)
DISC-08	CU-05, CU-08, CU-19	En modo nube (`ML_EN_NUBE=1`) no se deben escanear carpetas locales del servidor; en modo híbrido sí (`docs/PENDIENTES.md:212`, `docs/DESPLIEGUE.md:34`).	1) `frontend/src/views/HoyView.tsx:111-114` llama incondicionalmente a `api.escanearMaterial()` al montar la vista sin consultar `en_nube`, mostrando el bloque lateral «Material nuevo» (`HoyView.tsx:321-356`) incluso en Render. 2) En `frontend/src/views/MaterialView.tsx:297` y `539-626`, aunque se lee `enNube` y se muestra un banner informativo (`líneas 539-556`), no se oculta la tarjeta «Tus carpetas de asignatura» (`líneas 559-626`) ni se evita llamar a `api.escanearProyectos()` al montar (`línea 297`).	Media	No registrada (candidata nueva)
DISC-09	CU-10	Cerrar una tutoría debe marcar como completada la clase concreta (por su `id`), no otra clase que tenga el mismo nombre de tema (`docs/PENDIENTES.md:213-214`).	En la interfaz web se corrigió en P1 (`TutoriaView.tsx:173` envía `claseId`), pero en la CLI (`ml init`), `pedir_datos_inicio` (`backend/src/modular_learning/apps/cli/ui/interaccion.py:81-86, 104`) selecciona la clase del cronograma y devuelve solo `(materia, tema, subtemas_extra, enfoque)`, descartando `clase_seleccionada.get("id")`. Luego `apps/cli/commands/init.py:29-34` llama a `tutoria_service.iniciar_sesion` sin `clase_id`, haciendo que al finalizar se marque la primera clase cuyo `tema` coincida por texto (`core/domain/cronograma.py:55-65`).	Media	No registrada (P1 solo cubrió la web)
DISC-10	CU-03, CU-13	Los cortes de parcial definidos en la gestión de cronogramas (`SilabosView`) deben permitir seleccionar todas las clases de ese parcial al crear una guía offline en `GuiaView`.	En `frontend/src/views/GuiaView.tsx:346` y `360`, `seleccionarCorteParcial` busca la clase límite comparando el `id` de la clase: `actual.clases.findIndex((c) => String(c.id) === String(p.hasta))` (consistente con el docstring de `core/application/materias.py:155`: `[{'nombre': 'Parcial 1', 'hasta': 'clase_id'}]`). Sin embargo, en `frontend/src/views/SilabosView.tsx:666` y `708-711`, el formulario «Cortes de Parcial» pide una fecha en texto libre (`placeholder="Fecha límite (ej. 15/10/2026)"`) y la guarda en `hasta`. Por tanto, un corte de parcial creado desde `SilabosView` nunca coincide con `c.id` en `GuiaView`.	Media	No registrada (candidata nueva)

DISC-11	CU-01, CU-08, CU-09	Al eliminar un proyecto, eliminar una asignatura o vaciar materiales, todos los registros cruzados (`proyectos.json`, `registro_materiales.json`, cronogramas y notas en Obsidian) deben quedar consistentes, y las fichas de proyectos deben servir de contexto al tutor.	1) `ProyectosService.analizar_e_ingestar_proyecto` (`core/application/proyectos.py:154-158`) inserta una entrada `[PROYECTO] {nombre} ({parcial})` en `config/registro_materiales.json` y crea la nota en Obsidian, pero `ProyectosService.eliminar_proyecto` (`proyectos.py:55-56`) solo borra de `config/proyectos.json` (deja huérfana la entrada en `registro_materiales.json` y la nota `.md`). 2) `MateriasService.eliminar_materia` (`core/application/materias.py:56-67`) no elimina los proyectos de esa asignatura en `config/proyectos.json`. 3) `IngestaService.vaciar_materiales` (`core/application/ingesta.py:255-277`) no limpia `materiales_asociados`, `subtemas_adicionales` ni `material_transversal` dentro de `schedules/<slug>_cronograma.json`. 4) `TutoriaService` (`core/application/tutoria.py:148-151`) solo inyecta contexto desde `PluginsService`, ignorando las fichas de `ProyectosService`.	Media	No registrada (candidata nueva)
DISC-12	CU-17, CU-19	La documentación técnica (`README.md` y `docs/ARCHITECTURE.md`) debe coincidir con las rutas de almacenamiento, las métricas reales de `ml stats` y los comandos registrados en la CLI.	1) `docs/ARCHITECTURE.md:44, 91` ubica `materias.json` y `registro_materiales.json` en la raíz de `data/`, cuando en el código están bajo `data/config/` (`infrastructure/persistence/json_repositorios.py:41, 322`). 2) `README.md:116` y `docs/ARCHITECTURE.md:86` afirman que `ml stats` / `TelemetriaService` calculan «rachas de estudio», pero `TelemetriaService.obtener_estadisticas` (`core/application/telemetria.py:36-69`) no calcula ninguna racha. 3) `backend/src/modular_learning/apps/cli/commands/snapshot.py:1-125` no está registrado en `apps/cli/main.py` y realiza I/O directo (`os.walk`, `open`) fuera de los puertos hexagonales.	Baja	No registrada (candidata nueva)

6.2. Tabla de trazabilidad completa: Endpoints HTTP, Comandos CLI y Scripts → Casos de Uso

A. Endpoints HTTP del Backend (`backend/src/modular_learning/apps/api/`)

Están listados los **55 endpoints** expuestos por FastAPI (1 en `apps/api/main.py` y 54 distribuidos en los 11 routers de `apps/api/routers/`):

#	Método y Ruta HTTP	Router / Archivo (`archivo:línea`)	Servicio de Aplicación invocado (`archivo:línea`)	Caso de Uso
1	`GET /api/salud`	`apps/api/main.py:57-60`	Ninguno (healthcheck directo exento de auth)	**CU-19**
2	`GET /api/config`	`apps/api/routers/config.py:17-23`	`ConfiguracionService.obtener_ajustes` (`core/application/configuracion.py:25-26`)	**CU-18**
3	`POST /api/config`	`apps/api/routers/config.py:26-36`	`ConfiguracionService.guardar_ajustes` (`core/application/configuracion.py:28-31`)	**CU-18**
4	`GET /api/materias`	`apps/api/routers/materias.py:22-29`	`MateriasService.listar_materias` y `listar_catalogo` (`core/application/materias.py:30-36`)	**CU-01**
5	`POST /api/materias`	`apps/api/routers/materias.py:32-38`	`MateriasService.agregar_materia` (`core/application/materias.py:42-45`)	**CU-01**
6	`PUT /api/materias/{slug}`	`apps/api/routers/materias.py:41-51`	`MateriasService.renombrar_materia` (`core/application/materias.py:47-54`)	**CU-01**
7	`DELETE /api/materias/{slug}`	`apps/api/routers/materias.py:54-62`	`MateriasService.eliminar_materia` (`core/application/materias.py:56-67`)	**CU-01**
8	`GET /api/materias/cronogramas`	`apps/api/routers/materias.py:65-69`	`MateriasService.listar_cronogramas` (`core/application/materias.py:81-85`)	**CU-03**, **CU-19**
9	`GET /api/materias/{materia}/cronograma`	`apps/api/routers/materias.py:72-78`	`MateriasService.obtener_cronograma` (`core/application/materias.py:70-73`)	**CU-03**
10	`DELETE /api/materias/{materia}/cronograma`	`apps/api/routers/materias.py:81-89`	`MateriasService.eliminar_cronograma` (`core/application/materias.py:87-94`)	**CU-04**
11	`GET /api/materias/{materia}/clases`	`apps/api/routers/materias.py:92-97`	`MateriasService.obtener_proximas_clases` (`core/application/materias.py:75-79`)	**CU-03**, **CU-10**
12	`POST /api/materias/{materia}/clases`	`apps/api/routers/materias.py:100-110`	`MateriasService.agregar_clase` (`core/application/materias.py:96-108`)	**CU-03**

13	`PUT /api/materias/{materia}/clases/{clase_id}`	`apps/api/routers/materias.py:113-126`	`MateriasService.editar_clase` (`core/application/materias.py:110-124`)	**CU-03**
14	`DELETE /api/materias/{materia}/clases/{clase_id}`	`apps/api/routers/materias.py:129-138`	`MateriasService.eliminar_clase` (`core/application/materias.py:126-137`)	**CU-03**
15	`PATCH /api/materias/{materia}/clases/{clase_id}/estado`	`apps/api/routers/materias.py:141-151`	`MateriasService.cambiar_estado_clase` (`core/application/materias.py:139-143`)	**CU-03**
16	`GET /api/materias/{materia}/parciales`	`apps/api/routers/materias.py:154-159`	`MateriasService.obtener_parciales` (`core/application/materias.py:146-150`)	**CU-03**, **CU-13**
17	`POST /api/materias/{materia}/parciales`	`apps/api/routers/materias.py:162-172`	`MateriasService.guardar_parciales` (`core/application/materias.py:152-159`)	**CU-03**
18	`POST /api/silabo/extraer`	`apps/api/routers/silabo.py:23-50`	`SilaboService.sugerir_nombre_materia` y `extraer_cronograma` (`core/application/silabo.py:72-117`)	**CU-02**
19	`POST /api/silabo/guardar`	`apps/api/routers/silabo.py:53-67`	`SilaboService.guardar_cronograma` (`core/application/silabo.py:119-125`)	**CU-02**
20	`GET /api/material/registrados`	`apps/api/routers/material.py:26-30`	`IngestaService.listar_materiales_ingestados` (`core/application/ingesta.py:171-173`)	**CU-07**, **CU-01**
21	`POST /api/material/vaciar`	`apps/api/routers/material.py:33-39`	`IngestaService.vaciar_materiales` (`ingesta.py:255-277`) y `ProyectosService.vaciar_proyectos` (`proyectos.py:201-207`)	**CU-09**, **CU-18**
22	`DELETE /api/material/{nombre_archivo:path}`	`apps/api/routers/material.py:42-51`	`IngestaService.eliminar_material` (`core/application/ingesta.py:175-203`)	**CU-07**
23	`GET /api/material/escanear`	`apps/api/routers/material.py:54-61`	`IngestaService.escanear` / `escanear_materia` (`core/application/ingesta.py:205-253, 279-280`)	**CU-05**, **CU-19**
24	`POST /api/material/sincronizar/stream`	`apps/api/routers/material.py:64-84`	`IngestaService.ingestar_pendientes_stream` (`core/application/ingesta.py:282-323`)	**CU-05**
25	`POST /api/material/ingestar`	`apps/api/routers/material.py:87-118`	`MateriasService.agregar_materia` y `IngestaService.ingestar_archivo` (`core/application/ingesta.py:104-169`)	**CU-06**
26	`GET /api/proyectos`	`apps/api/routers/proyectos.py:23-29`	`ProyectosService.listar_proyectos` (`core/application/proyectos.py:48-50`)	**CU-08**
27	`GET /api/proyectos/escanear`	`apps/api/routers/proyectos.py:32-48`	`ProyectosService.escanear_proyectos_en_carpeta` y `verificar_estado_proyecto` (`core/application/proyectos.py:58-66, 161-186`)	**CU-08**
28	`GET /api/proyectos/{id_proyecto:path}/estado`	`apps/api/routers/proyectos.py:51-59`	`ProyectosService.verificar_estado_proyecto` (`core/application/proyectos.py:161-186`)	**CU-08**
29	`GET /api/proyectos/{id_proyecto:path}`	`apps/api/routers/proyectos.py:62-70`	`ProyectosService.obtener_proyecto` (`core/application/proyectos.py:52-53`)	**CU-08**
30	`POST /api/proyectos/ingestar`	`apps/api/routers/proyectos.py:73-88`	`ProyectosService.analizar_e_ingestar_proyecto` (`core/application/proyectos.py:118-159`)	**CU-08**
31	`POST /api/proyectos/{id_proyecto:path}/actualizar`	`apps/api/routers/proyectos.py:91-102`	`ProyectosService.actualizar_proyecto` (`core/application/proyectos.py:188-199`)	**CU-08**
32	`DELETE /api/proyectos/{id_proyecto:path}`	`apps/api/routers/proyectos.py:105-113`	`ProyectosService.eliminar_proyecto` (`core/application/proyectos.py:55-56`)	**CU-08**
33	`POST /api/tutoria/iniciar`	`apps/api/routers/tutoria.py:33-69`	`MateriasService.agregar_materia` y `TutoriaService.iniciar_sesion` (`core/application/tutoria.py:115-165`)	**CU-10**
34	`GET /api/tutoria/{sesion_id}/paso/{indice}/stream`	`apps/api/routers/tutoria.py:72-98`	`TutoriaService.explicar_paso_stream` (`core/application/tutoria.py:225-238`)	**CU-10**
35	`POST /api/tutoria/{sesion_id}/duda`	`apps/api/routers/tutoria.py:101-115`	`TutoriaService.responder_duda` (`core/application/tutoria.py:240-253`)	**CU-10**
36	`POST /api/tutoria/{sesion_id}/finalizar`	`apps/api/routers/tutoria.py:118-130`	`TutoriaService.finalizar_sesion` (`core/application/tutoria.py:255-281`)	**CU-10**

4	`ml quiz`	`apps/cli/main.py:44-49`	`apps/cli/commands/quiz.py:14-79`	`MateriasService.listar_materias`, `EvaluacionService` (`generar_quiz`, `evaluar_respuestas`)	**CU-11**
5	`ml stats`	`apps/cli/main.py:52-57`	`apps/cli/commands/stats.py:15-68`	`TelemetriaService.obtener_estadisticas`	**CU-19**
6	`ml up`	`apps/cli/main.py:60-65`	`apps/cli/commands/up.py:15-54`	`PluginsService` (`obtener_plugins`, `actualizar_plugin`)	**CU-17**
7	`python -m modular_learning.apps.cli.commands.snapshot`	`apps/cli/commands/snapshot.py:124-125` (no registrado en `apps/cli/main.py`)	`apps/cli/commands/snapshot.py:10-125`	`MateriasService.listar_materias`, `PluginsService` (`obtener_plugin_para_materia`, `guardar_contexto_extraido`)	**CU-17**
8	`ml-web`	`backend/pyproject.toml:34`	`apps/api/main.py:135-141`	Servidor FastAPI / Uvicorn (expone todos los endpoints HTTP #1 a #55)	**CU-01** a **CU-19**
9	`python scripts/migrar_datos.py [--simular]`	`scripts/migrar_datos.py:239-240`	`scripts/migrar_datos.py:23-236`	`crear_almacen` (`AlmacenDocumentos`), `Materia`, `asegurar_ids_clases`	**CU-20**
10	`python deploy/subir_datos.py [--subir]`	`deploy/subir_datos.py:75-76`	`deploy/subir_datos.py:45-73`	`AlmacenArchivos`, `AlmacenPostgres`	**CU-20**